

UMEDCA

Theodore M. Savich

September 5, 2025

Contents

1	Operation	5
1.1	Introduction: From Numbers to Operations	5
1.2	The Constructivist Context and the Motivation for Formalization .	6
1.2.1	CGI and the Constructivist Tradition	6
1.2.2	The Challenges of Scaling Constructivism	7
1.2.3	The Necessity of Formalization	7
1.3	The Hermeneutic Calculator: Choreography as Computation . . .	8
1.4	Embodied Metaphors and Material Inference	9
1.5	The Sliding Strategy: A Case Study in Mathematical Development	10
1.6	From Embodied Practice to Formal Structure	11
1.7	Inversion and the Dialectical Structure of Operations	12
1.8	Critical Arithmetic and Mathematical Meaning-Making	13
2	Algorithmic Elaboration and History	15
2.1	Introduction: History as Algorithm and Dialectic	16
2.2	An Instance of Algorithmic Elaboration	17
2.2.1	Euclid’s Proof: An Uncontainable Infinity	19
2.2.2	An Algorithmic Elaboration of Euclid to Incompatibility Semantics	21
2.2.3	A Formal Reconstruction in Incompatibility Semantics . .	22
2.3	Algorithmic Elaboration and Rules of Mathematical Inference . .	26
2.4	The Limitations of Algorithmic Elaboration: Dialectical Moments	31
2.4.1	Brandom’s Pragmatic Expressive Bootstrapping	31
2.5	Formalizing Bootstrapping Procedures	33
2.5.1	Reflecting on the Materiality of Mathematical Inferences .	35
2.6	A Next Step: Hylomorphism and Poetic Computation	36
2.6.1	Homoiconicity as the Key	36
2.7	Conclusion	37

3	Operation	39
3.1	Introduction: From Numbers to Operations	40
3.2	Deconstructing Operations: The Case of Multiplication	42
3.2.1	The Embodied Ground: Subjective Validity	43
3.2.2	The Recipe: From Embodiment to Norms	43
3.2.3	The Phenomenological Ground of Operation	44
3.2.4	The Embodied Ground of Norms: Transposing Lakoff and Núñez	47
3.3	The Subtlety of Sameness: Commutativity and Structural Trans- formation	50
3.3.1	A Formal Glimpse: The Paradox of Internalized Commu- tation	50
3.3.2	The Emergent Automaton: Objective Validity	51
3.4	Conclusion: Operations as a Triadic Unity	52
3.5	Appendix: The Hermeneutic Calculator	52
3.5.1	Choreography as Computation	53
3.5.2	Two Fundamental Movements	53
3.5.3	Fractal Architecture: Iterative Core + Strategic Shell	53
3.5.4	The Formal Model	54
3.5.5	Mechanisms of Elaboration	55
3.5.6	Inversion of Practice	56
3.5.7	Collaboration and Iterative Refinement	56
3.5.8	Why Executable Formal Models Matter	56
3.5.9	Algorithmic Elaboration (Brandom Frame)	57
3.5.10	Scope of This Document	57
3.6	Hermeneutic Calculator: Strategy Formalizations	57
3.7	Counting and Counting On	58
3.8	Rearranging to Make Bases (RMB)	59
3.9	COBO (Counting On by Bases then Ones)	60
3.10	Rounding and Adjusting (Addition)	61
3.11	Chunking (Addition)	62
3.12	Subtraction Chunking (Three Orientations)	63
3.13	Subtraction COBO / CBBO	65
3.14	Subtraction Decomposition (Borrowing)	66
3.15	Subtraction Rounding and Adjusting	67
3.16	Subtraction Sliding (Constant Difference)	68
3.17	Commutative Reasoning (Multiplication Optimization)	69
3.18	Coordinating Two Counts (C2C)	70

3.19	Conversion to Bases and Ones (CBO Multiplication)	70
3.20	Distributive Reasoning (Multiplication)	71
3.21	Dealing by Ones (Division – Sharing)	72
3.22	Inverse Distributive Reasoning (Division)	72
3.23	Using Commutative Reasoning (Division via Iterated Accumulation)	73
3.24	Conversion to Groups Other than Bases (CFOB Division)	74
3.25	Conceptual Dependency Graph (Narrative)	75
3.26	Temporal Dynamics Summary	75
3.27	Glossary of Symbols	75
3.28	Fractions	76
3.29	The Operational Foundation: The Explicitly Nested Number Sequence (ENS)	77
3.30	The Emergence of Fractional Schemes	77
3.30.1	The Partitive Fractional Scheme (PFS)	78
3.30.2	Metamorphic Accommodation: Recursive Partitioning	78
3.31	The Formal Automaton Model	79
3.31.1	Python Implementation	79
3.31.2	Execution and Analysis	79
3.32	Conclusion: From Psychological Description to Pragmatic Specification	80
4	9: Algorithmic Elaboration and History	83
4.1	The Misrecognition of Totality	84
4.2	The First Determinate Negation: The Inferential Turn	85
4.3	Systematic Analysis: Euclid’s Proof Reconstructed	86
4.4	The Dialectical Turn: Limits and Embodied Foundations	87
4.5	Distinguishing Two Modes of Practice Development: PEdT and AE	89
4.6	Second-Person Reflection: Pragmatic Expressive Bootstrapping	94
4.7	Integration: Modeling Conceptual Change	96
4.8	Conclusion: History as Becoming	97
5	10: Operation	101
5.1	The Misrecognition of Operation	102
5.2	The First Determinate Negation: Critical Arithmetic and the Triad	103
5.3	Systematic Analysis: The Recipe from Embodiment to Norms	106
5.4	The Dialectical Turn: Choreography and Fractal Geometry	108
5.5	Second-Person Reflection: Elaboration and Formalization	109
5.6	Integration: The Dialectical Structure of Operations	114

5.7 Conclusion: Operations as Creative Possibility	115
--	-----

Chapter 1

Operation

Abstract

This chapter explores mathematical operations as embodied, normatively regulated practices that emerge from our engagement with the world. Developing a unified theoretical framework that synthesizes embodied metaphors, material inferentialism, and student-invented strategies, the investigation reveals how formal arithmetic structures emerge from lived experience. The chapter articulates a “critical arithmetic” that captures the dynamic nature of mathematical operations as expressions of our being-in-the-world, grounded in material interaction, regulated through social practice, and crystallized into reliable formal procedures. Through analysis of the Hermeneutic Calculator project, which formalizes twenty-five student strategies as computational choreographies, the work demonstrates how sophisticated mathematical understanding develops through the systematic transformation of simpler practices. This approach challenges conventional accounts that treat operations as mechanical symbol manipulation, instead revealing arithmetic as an open field of creative possibility where students participate in the ongoing elaboration of mathematical meaning-making.

1.1 Introduction: From Numbers to Operations

The preceding chapters reconstructed the nature of number itself, beginning with a student’s profound question: “Mr. Savich, what even is two?” That moment of

grief and frustration opened a path away from viewing numbers as abstract, pre-existing objects toward understanding them as first-person recollective pronouns that emerge from the dynamic interplay of linguistic self-reference.

Having explored what numbers *are* – reflections of dynamic, self-recollecting human being – this chapter now turns to the equally fundamental question of what it means to *do* things with them. The investigation reveals mathematical operations not as formal manipulations of abstract symbols but as expressions of embodied, normatively regulated practices that emerge from our engagement with the world and with others.

The chapter develops a unified theoretical framework that synthesizes three complementary perspectives: the embodied metaphors of Lakoff and Núñez (**Lakoff2000**), Brandom’s material inferentialism (**Brandom2008**), and the empirical landscape of student-invented strategies documented by Cognitively Guided Instruction research (**Carpenter1999**). This synthesis articulates what I call “critical arithmetic” – an open-ended, pre-formal system that captures the dynamic nature of mathematical operations as they unfold in lived experience.

1.2 The Constructivist Context and the Motivation for Formalization

The investigation into student-invented strategies is deeply indebted to the paradigm shift initiated by Cognitively Guided Instruction (CGI). To fully appreciate the motivation for the formalization I undertake, it is crucial to understand the historical trajectory and the inherent limitations of the methodologies that first brought these strategies to light.

1.2.1 CGI and the Constructivist Tradition

Emerging in the 1980s from research by Carpenter, Fennema, and colleagues, CGI established that children possess rich, informal mathematical knowledge long before formal instruction (**WCER_CGI_Ingenuity**). This research provided the empirical foundation for the constructivist turn in mathematics education, asserting that students actively construct understanding by solving problems in meaningful ways, rather than passively receiving algorithms (**NU_Constructivism_Education**).

CGI operationalized the constructivist theories of Piaget, emphasizing assimilation and accommodation as the engines of cognitive development (**UBuffalo_Constructivism**).

1.2. THE CONSTRUCTIVIST CONTEXT AND THE MOTIVATION FOR FORMALIZATION⁹

By centering instruction on student-generated or “invented” strategies, CGI fundamentally altered the teacher’s role from instructor to facilitator, tasked with interpreting and building upon student thinking (**WCER_CGI_Ingenuity**). This approach provided a concrete pedagogical model for the vision articulated in the NCTM 1989 *Standards*, moving away from the rote memorization that characterized much of 20th-century instruction toward conceptual understanding and problem-solving (**Purdue_Dewey_NCTM**).

1.2.2 The Challenges of Scaling Constructivism

While CGI has demonstrated significant positive effects on students’ conceptual understanding (**WCER_CGI_Ingenuity**), its implementation presents substantial challenges that limit its scalability and sustainability. The core issue lies in the immense demand CGI places on teachers’ pedagogical and mathematical content knowledge. Effectively implementing CGI requires teachers to anticipate a wide range of student strategies, interpret unconventional reasoning in real-time, and make nuanced, responsive instructional decisions to guide diverse learners (**Kennesaw_CGI_Implementation**).

Furthermore, the student-centered, discourse-intensive nature of CGI is time-consuming. The time required for planning, allowing students to grapple with problems, and facilitating lengthy discussions often conflicts with systemic pressures from standardized curricula and pacing guides (**Kennesaw_CGI_Implementation; Hamline_CGI_Curriculum**). The flexible nature of the framework, while a philosophical strength, makes it difficult to ensure fidelity of implementation across different contexts (**TaylorFrancis_CGI_Review**). These practical constraints often create an environment hostile to the deep, responsive teaching CGI demands.

1.2.3 The Necessity of Formalization

These challenges motivate the need for a more structured understanding of the landscape of student-invented strategies. Within some constructivist circles, there exists a tension regarding formalization, sometimes perceived as an imposition of external structure—perhaps even a kind of violence against the cherished principle of student-centered invention. However, this perspective overlooks a crucial reality: not all invented strategies are equally efficient, generalizable, or mathematically productive.

Consider an analogy to protein folding. While there are billions of possible configurations a protein can adopt, only a small fraction result in viable, functional

structures. Similarly, while the possibilities for arithmetic manipulation between 1 and 1000 are vast, the number of strategies that actually make computation more efficient and reveal deeper mathematical structure appears to be limited.

The formalization project undertaken here, exemplified by the Hermeneutic Calculator, is not intended to restrict student invention but to rigorously analyze the strategies that emerge naturally. By treating these strategies as computational choreographies, we can move beyond the limitations of isolated teaching experiments and discover relationships that might otherwise remain obscured.

For example, the formal automata analysis reveals a precise fractal geometry underlying arithmetic development. It demonstrates how strategies are computationally related through mechanisms like inversion, compression, and nesting. The analysis identifies a fundamental **Iterative Core** (Initialize $\rightarrow$ Step $\rightarrow$ Check) that is embedded within more complex **Strategic Shells**, which optimize the iteration by analyzing the problem space (e.g., calculating a dynamic gap K). This computational self-similarity is visualized in Figure 5.1.

This formalization allows us to articulate exactly how sophisticated strategies emerge from simpler practices through **Algorithmic Elaboration (AE)**, distinguishing this logical restructuring from the pedagogical stabilization achieved through **Practical Elaboration by Training (PEdT)**. By making the structure of student thinking explicit, formalization provides a powerful tool for advancing the constructivist goal of deep conceptual understanding. It offers a map of the developmental terrain that can help teachers guide students more effectively, potentially addressing the challenges of scalability and teacher knowledge by revealing the objective, normative structures that emerge from embodied practice and stabilize around points of self-certainty (Figure 1.3).

1.3 The Hermeneutic Calculator: Choreography as Computation

Mathematical operations are, fundamentally, choreographed movements of thought. Each arithmetic strategy represents what I term “written choreography for embodied cognition” (**savich_hermeneutic_nodate**). A formal automaton becomes a script for the temporal unfolding of mathematical understanding: initialize, transform, check, recurse, terminate. This framing preserves *how* a student actually moves through a calculation – counting up, pausing at a boundary, decomposing a number – rather than replacing those movements with opaque symbolic shortcuts.

The analysis through the Hermeneutic Calculator project reveals two fundamental movements underlying all arithmetic strategies. First, **temporal compression** (sublation/recollection) unitizes many micro-acts into larger cognitive units: ten ones become one ten, three base jumps become a single composite stride. This compression accelerates mathematical flow. Second, **temporal decompression** (determinate negation) strategically expands a composite to restore fine control: borrowing a ten, splitting five into two plus three, decomposing a factor for distributive reasoning. Mathematical fluency emerges as students learn to coordinate these movements, developing the capacity to know *when* to expand and *when* to re-compress.

Across twenty-five formalized student strategies, I consistently recover the same fractal pattern. Each strategy exhibits an **iterative core** – a minimal loop of initialize, step, and condition check – nested within a **strategic shell** that prepares, optimizes, or transforms the problem so the core runs with fewer or cognitively lighter iterations. Strategies such as Rearranging to Make Bases, Sliding, and Distributive Reasoning wrap the iterative core with analysis that computes gaps, splits factors, or slides both numbers to maintain invariant relationships. Because the shell often invokes the core as a subroutine, the global structure becomes genuinely self-similar: strategies contain and sometimes nest earlier strategies, producing a computational fractal rather than mere metaphorical flourish.

1.4 Embodied Metaphors and Material Inference

Mathematical concepts emerge from the material reality of embodied interaction with the world. Lakoff and Núñez (**Lakoff2000**) identify four grounding metaphors that structure arithmetic understanding: Object Collection, Object Construction, Measuring Stick, and Motion Along a Path. These are not merely abstract mappings but embodied schemas that carry material inferences – inferences that are good not by virtue of logical form but by virtue of the content of their concepts (**Brandom2008**).

The *Arithmetic as Object Collection* metaphor, for instance, materially entails commutativity and associativity because the physical act of pooling collections is indifferent to the order of operations. When a child understands addition through gathering objects, the mathematical properties emerge from the material constraints of embodied activity. Similarly, the *Motion Along a Path* metaphor grounds subtraction in the experience of moving backward, creating the inferential structure that underlies strategies like Counting Back and Sliding.

However, these embodied origins require pedagogical stabilization to become reliable mathematical practices. This occurs through what I term **Practical Elaboration by Training** (PEdT) – the contingent, pedagogical process emphasized in Cognitively Guided Instruction that transforms implicit material inferences into explicit, teachable procedures. Only after this stabilization can the more sophisticated process of **Algorithmic Elaboration** (AE) occur, where existing practices are logically restructured into new, more powerful strategies.

1.5 The Sliding Strategy: A Case Study in Mathematical Development

Consider the Sliding strategy, which students invent to solve subtraction problems by maintaining constant differences. Faced with a calculation like $74 - 36$, a student might recognize that both numbers can be adjusted by the same amount without changing the difference: $(74 + 4) - (36 + 4) = 78 - 40 = 38$. This strategy exemplifies the developmental trajectory from embodied metaphor through normative regulation to formal structure.

The strategy emerges from the *Motion Along a Path* metaphor, where subtraction is understood as measuring the distance between two points. The material inference carried by this metaphor is that distances remain invariant under translation – if both endpoints move the same amount in the same direction, the distance between them is preserved. This insight becomes pedagogically stabilized when students learn to recognize when such transformations are beneficial and how to execute them systematically.

The formal automaton for Sliding reveals its computational structure: the strategic shell identifies a convenient adjustment value K that will simplify the calculation (often by creating a multiple of ten), then invokes the iterative core to perform the adjusted subtraction. The analysis phase – finding the gap K that transforms the subtrahend into a base-ten number – demonstrates the shell’s supervisory role over the core’s execution.

Figure 1.1: *The Geometry of Inversion: Addition and Subtraction as a Unified Structure*. The Möbius strip visualization articulates the dialectical relationship between addition and subtraction. The automata analysis demonstrates how they represent inverse orientations of the same underlying temporal structure.

Figure 1.2: *The Fractal and Dialectical Geometry of Arithmetic*. Fractalized Arithmetic as Automata: The fractal scaling of the Iterative Core demonstrates computational self-similarity. The superimposed loops show that the Core's structure remains invariant while the scale of action changes from units to bases. The dialectical inversion (Red/Purple) visualizes how the same structure operates in opposite orientations.

The fractal architecture illustrated in Figure 5.1 reveals how mathematical strategies achieve their power through self-similar structure. The iterative core – the fundamental loop of initialize, step, and check – scales across different mathematical magnitudes while maintaining its essential form. This scaling property allows strategies to work across place values, demonstrating how mathematical understanding exhibits genuine computational self-similarity rather than merely metaphorical resemblance.

Figure 1.3: *The Inference Field: Automata as Orbits Around Self-Certainty*. The inference field visualization articulates how mathematical automata operate as dynamic orbits around points of self-certainty. Each strategy traces a path through the space of mathematical possibilities, with stable attractors representing moments of cognitive equilibrium where understanding crystallizes into reliable practice.

1.6 From Embodied Practice to Formal Structure

The trajectory from embodied metaphor to formal mathematical structure occurs through two distinct but complementary elaboration mechanisms. Practical Elaboration by Training (PEdT) stabilizes the material inferences carried by embodied metaphors into reliable primitive practices. Students learn to count on consistently, to decompose numbers strategically, and to recognize when particular approaches are appropriate. This stabilization is fundamentally pedagogical – it depends on teaching, practice, and the social regulation of mathematical activity.

Algorithmic Elaboration (AE), by contrast, represents the logical restructuring of existing abilities rather than their mere strengthening through repetition. When students reorganize their Counting On practice into the more sophisticated Rearranging to Make Bases strategy, they are not simply becoming faster counters but are discovering new inferential relationships. The elaborated strategy makes ex-

plicit what was implicit in the original practice: that numerical relationships can be preserved under certain transformations.

Strategies that stand in an LX (Elaborated-Explicating) relationship – where later practices both derive from and make explicit earlier ones – constitute what mathematics educators recognize as “conceptual understanding”. The Rearranging to Make Bases strategy explicates the associative structure latent in Object Collection. Distributive Reasoning articulates the multiplicative structure implicit in repeated addition. These relationships demonstrate how formal mathematical properties emerge from and remain grounded in embodied practice.

1.7 Inversion and the Dialectical Structure of Operations

The analysis reveals that mathematical operations exhibit a fundamentally dialectical structure. Subtraction strategies emerge not through the invention of entirely new procedures but through the inversion or repurposing of addition approaches. The Missing Addend strategy reframes subtraction as forward accumulation. Counting Back mirrors Counting On. The Sliding strategy preserves difference across translation, while Borrowing reverses the carry procedure by decompressing previously sublated place-value units.

This dialectical relationship extends beyond the superficial observation that subtraction “undoes” addition. The formal automata reveal that operations often share identical computational cores while differing in their orientations or interpretations. The Möbius strip visualization in Figure 5.2 captures this relationship: the underlying structure remains invariant while the operational orientation flips between additive and subtractive modes.

The inference field depicted in Figure 1.3 provides a dynamic understanding of how mathematical strategies operate as orbits around points of self-certainty. Each calculation traces a path through the space of mathematical possibilities, with strategies serving as attractor basins that guide mathematical thinking toward stable, repeatable procedures. The field structure reveals how individual strategies are not isolated techniques but elements within a larger ecology of mathematical understanding.

1.8 Critical Arithmetic and Mathematical Meaning-Making

The investigation into mathematical operations as embodied, normatively regulated practices challenges conventional accounts that treat arithmetic as the mechanical manipulation of formal symbols. Instead, operations emerge as expressions of our being-in-the-world, grounded in material interaction, regulated through social practice, and crystallized into reliable formal procedures.

This understanding has profound implications for mathematical education. Rather than presenting operations as arbitrary rules to be memorized, pedagogy can build on the natural elaboration mechanisms through which students develop mathematical understanding. The recognition that sophisticated strategies emerge through the systematic transformation of simpler practices suggests that mathematical development follows discoverable patterns rather than unpredictable leaps.

The Hermeneutic Calculator project demonstrates that student-invented strategies can be formalized without losing their connection to embodied meaning. The automata serve not as replacements for human mathematical thinking but as precise descriptions of the temporal unfolding of mathematical understanding. By treating each strategy as choreographed movement of thought, the formalization preserves rather than eliminates the human dimension of mathematical activity.

The critical arithmetic developed in this chapter offers an alternative to both mechanical algorithmism and romantic constructivism. Mathematical operations are neither arbitrary formal procedures nor purely subjective constructions but rather expressions of the dialectical relationship between embodied experience and normative structure. They emerge from material interaction with the world, become stabilized through social practice, and achieve objective validity through their capacity to generate reliable, repeatable results.

Understanding mathematical operations as embodied, normatively regulated practices reveals arithmetic not as a closed system of fixed procedures but as an open field of creative possibility. Students do not simply learn to execute predetermined algorithms but participate in the ongoing elaboration of mathematical understanding. This participation is simultaneously deeply personal – grounded in each individual's embodied experience – and thoroughly social – regulated through the shared norms of mathematical practice.

The analysis demonstrates how formal mathematical structures emerge from rather than supersede lived experience. The automata that model student strategies are not abstract computational devices but precise articulations of embodied

mathematical thinking. They reveal the temporal dynamics through which mathematical understanding unfolds, showing how the formal and the felt, the abstract and the concrete, the subjective and the objective are integrated within a unified account of mathematical meaning-making.

Chapter 2

Algorithmic Elaboration and History

Abstract

This chapter explores the development of mathematical concepts and procedures through the lens of algorithmic elaboration, arguing that mathematical history reflects a dialectical pattern of apparent completion generating new possibilities. The chapter connects the reconceptualization of numerals as pronouns with the development of critical arithmetic, demonstrating how Robert Brandom's concept of algorithmic elaboration models this evolution. Two case studies anchor the analysis: a reconstruction of Euclid's proof of the infinity of primes using incompatibility semantics, and an examination of how arithmetic operations emerge from and transcend embodied metaphors. This analysis reveals the limitations of simple algorithmic elaboration, highlighting the need for "pragmatic expressive bootstrapping," where conceptual systems explicate their own implicit normative structures. The chapter also challenges traditional approaches to literature review, advocating for understanding intellectual history as a dynamic process of conceptual development mirroring algorithmic elaboration within mathematical practice. This lays the groundwork for a subsequent exploration of critical arithmetic, illustrating how operations emerge from embodied practices through normatively regulated elaboration. The chapter draws upon the work of Brandom, Euclid, and Lakoff and Núñez.

2.1 Introduction: History as Algorithm and Dialectic

Traditionally conceived, algorithms are step-by-step procedures for solving problems or performing tasks. They are foundational to computer science, mathematics, and many areas of human activity. An algorithm takes an input, processes it through a series of defined steps, and produces an output. What this definition of algorithms can miss is the underlying inferential structure of algorithms. Essentially, an algorithm is a chain of inferences, as discussed in section ???. These chains of inference can be smoothed, just like someone who practices yoga can learn to flow between the poses. Of particular interest are chains of ??? involving substitution. The concept of algorithmic elaboration was also already introduced as Brandom does, in figure ??, where the long division algorithm is elaborated from the practices of multiplication and subtraction.

When I proposed this book, I was interested in exploring the concept of literature reviews. In the intervening years, I wrote enough on that idea to fill a book and sated my curiosity about the topic. The Bridge chapter does what I intended to do in this chapter by explaining how the reification of a totality in a name inevitably leads to an expression of that totality that cannot have been in the reified totality but must have been in the original totality. This process is at work when conceptualizing “The Literature.”

I found literature reviews to be terribly confusing when I was writing my dissertation. As Boote and Beile note, “The perceived lack of importance of the dissertation literature review is seen in the paucity of research and publications devoted to understanding it” (boote2005). I thought it would be a gift to ink some prose devoted to understanding the process of literature review. The earth shifted under my feet, as the popularization of Large Language Models seem to make the processes of summarizing and synthesizing a more or less passive activity. However, as a practical matter, I still do not know how to do a literature review. Why try to advise when I do not know? It is not that I do not care what scholars have said before me, but I could not help but feel like every time I tried to gain some purchase on the totality, I found a new gap. Some other article would fill that gap, and then I would find another, and another, and another. I had not yet synthesized the groundless ground, and it felt like I was trying to climb onto the shoulders of giants who were made of sand.

As I continued to write on this project, I decided that I would not write about literature reviews in the way I originally intended, but I still was not sure how to

approach the topic of history and the concept of algorithmic elaboration. Instead, I will demonstrate the concept of history as an algorithmic elaboration by instantiating it within the history of mathematics. I will then reconstruct Euclid’s proof from the Bridge. I will then discuss the limits of algorithmic elaboration, particularly in the context of dialectical moments in the history of mathematics. The way I articulated diagonalization in the Bridge can be elaborated into Brandom’s notion of *pragmatic expressive bootstrapping*. That notion will help justify how the “no” of determinate negation (recall ??) is sufficient to explicate a full mathematical system.

This introduction reveals a crucial methodological insight about the relationship between academic literature review and mathematical development. The traditional approach to literature review – attempting to master “The Literature” as a static totality – reproduces the same reification problem explored in the Bridge chapter through Cantor’s diagonalization. Just as any attempt to completely enumerate mathematical infinities generates elements that exceed the enumeration, any attempt to completely survey intellectual history encounters the dynamic, self-transforming nature of ideas themselves. Rather than treating concepts as fixed objects to be catalogued, this chapter will demonstrate how mathematical history unfolds through algorithmic elaboration – a process where implicit practical abilities become explicit through chains of material inference. This shift from static literature review to dynamic conceptual development prepares readers to see how Euclid’s ancient proof can be reconstructed using contemporary logical frameworks, revealing the ongoing, self-elaborating character of mathematical understanding itself.

2.2 An Instance of Algorithmic Elaboration

Since I already discussed the bones of Euclid’s proof, I will begin my reconstruction by introducing some terms from Brandom’s *incompatibility semantics* (Brandom2008). The first is the notion of an *incoherence frame*. An *incoherence frame* is a pair

$$\langle L, Inc \rangle,$$

where *Inc* specifies which sentences in the language *L* are *incoherent*. For Brandom, these are formal languages written by automata. But it does not do much harm to the notion to think of a language as a collection of sentences a qualitative researcher seeks to understand, or the way that people in general reflect on

troubling interactions to try to make sense of them. Incoherence can be existentially terrifying. When I am recognized as someone who believes x and $\neg x$, the recognizer might understand my beliefs as evolving. I can be taken as being who is becoming. But I can also be taken as someone who cannot be trusted. There is a risk of duplicity in incoherence. Context matters. When I speak with the kids about a stuffed toy dog, the existential significance of incoherence in calling the toy a mammal is not a shifting terror. But when I commit myself to incoherence in the academic field of “The Literature,” as I have done in the past and fear I will do again, the riskiness of commitment is much more apparent.

To comply with Brandom’s formalization of *incompatibility*, consider an incoherent set such as A : “now is night” and B : “now is day,” where both occurrences of “now” are intended to refer to the same moment (the same token reiterated, not merely the same type). By the *persistence* axiom, simply adding more sentences to the language does not repair this incoherence.

That is, there is no C that would allow for $A \cup B \cup C$ to be coherent. I am not sure that rule is generally true, though it makes sense for the somewhat inert sentences formed by automata. When each instance of language use is recognized as an action with an inherently reflective aspect, saying one more thing transforms what came before. A more general point: once a commitment to incoherence is recognized, attempting to wield it as domination by refusing critique is probably a mistake.

In formal mathematics, those contradictory commitments are dropped as falsities. In mathematics education, the tensions of misrecognition (‘errors,’ ‘mistakes,’ ‘misconceptions’) are sublated into a new understanding. They do not just ‘go away.’

An *incompatibility relation* I is defined so that two sets X and Y are *incompatible* if and only if their union is incoherent. That is, A is incompatible with B if taking them together results in an incoherence. Symbolically, $A \in I(B)$ iff $A \cup B \in Inc$.

Following Brandom’s notion, acceptance can be framed either in terms of truth (alethic modality) or morality (deontic-normative modality), and the assessment of an inference can be expressed as whether the consequent is incompatible with everything that the antecedent is incompatible with. A sentence like A : “Pokey is a mammal” is a looser claim than B : “Pokey is a dog,” but everything incompatible with “is a mammal” is also incompatible with “is a dog.” Sort of. This assumes shared background knowledge (e.g., that all dogs are mammals). A stuffed toy doggy is not a mammal. Brandom would say *B incompatibility entails A* and write: $(B \models_I A)$.

I will use a few more symbols that Brandom develops that relate to modal logic. There are many different modal logics. The idea is that when speakers make moral claims, truth claims, temporal claims etc., each makes explicit some rule or norm. Those rules differ depending on the category that is foregrounded in the claim. Using modal logic allows for some of those nuances to be captured, but they do not repair the fact that speech slips between categories in empirical dialog. A temporal claim habitually transforms into a spatial claim when it is recollected. The Kantian categories of understanding are inside and outside of each other, at least for the critical ethnographer. The rigidity implied by a foreign alphabet should not be taken to detract from core insights based on lived experiences: the rules of speech are not rigid.

- Np for negation (“not p ”),
- Kpq for conjunction (“ p and q ”),
- Apq as shorthand for $NK(Np)(Nq)$ (the disjunction “ p or q ”),
- Cpq as shorthand for $NKp(Nq)$ (the conditional “if p then q ”),
- Lp for a *necessity* operator (introduced via incompatibility semantics), and
- $Mp := NLNp$ for *possibility* (dually).

2.2.1 Euclid’s Proof: An Uncontainable Infinity

Euclid of Alexandria (circa 300 BCE) claims in Book 9, Proposition 20 of *The Elements* (euclid_euclids_2007) that any finite list of prime numbers is incomplete. This is often misrepresented when it is reconstructed. I recognize the experience below as my own:

Once upon a time a learned professor explained to a class of bright students how Euclid proved the existence of infinitely many prime numbers in the third century BC. He said that Euclid began by supposing only finitely many prime numbers exist and ultimately deduced a contradiction. He also remarked on the admirable simplicity of what he said was Euclid’s proof by contradiction. He explained that the proof began by letting $p_1 \dots p_n$ be all the prime numbers that exist, in increasing order. He directed the students’ attention to the number $p_1 \dots p_n + 1$. He showed that that number cannot be divisible by any

of the primes $p_1, \dots, p_n$: Then he said, ‘By our assumption, no other primes than those exist. This number is therefore not divisible by any primes. Since it is not divisible by any primes, it must itself be prime. But that contradicts our initial assumption that no other primes than $p_1, \dots, p_n$ exist.’(hardy2009prime)

A more faithful reconstruction of Euclid follows. One aspect of the common misrepresentation of Euclid’s work is that Euclid provides an algorithm for finding primes. This is not true. It is true that multiplying *all* the known primes together and adding one yields a number that is not divisible by any of the known primes. However, Euclid does not conduct that multiplication or claim that the result is prime. $2 \times 3 \times 5 + 1$ is prime, but $2 \times 3 \times 5 \times 7 \times 11 \times 13 + 1 = 30031 = 59 \times 509$. He only claims that it is not divisible by any of the known primes. The number could be composite, and it could have prime factors that are not in the original list.

The proof unfolds within the conceptual framework of ancient Greek mathematics, using notions of length, measurement, and the incommensurability of certain magnitudes. Rather than using modern algebraic notation, Euclid speaks of lengths that cannot be measured by each other without remainder (primes), and those that can be measured by those lengths (composites). One possible translation of his proof is as follows:

Let the prime numbers which have been given be A , B , and C . I say that the prime numbers are more numerous than A , B , and C . For let us take the least number which is measured by A , B , and C , and let that number be denoted by DE , and let the unit be appended to DE , forming the number EF .

Now, either EF is prime or it is not. Suppose first that EF is prime. Then the collection of prime numbers A , B , C , and EF is larger than the collection A , B , and C .

But if EF is not prime, then it is measured by some prime number, say G . And G cannot be identical with any of A , B , or C , for if it were, then (since A , B , and C measure DE) G would also measure EF , and further the unit would be measured by G – which is absurd.

Therefore, in this case as well, the prime numbers A , B , C , and G are more numerous than the assigned set A , B , and C . This is what was required to be demonstrated.

Figure

Figure 2.1: *Note.* Cuisenaire rods, interpreted as the numerals 1-10, building a composite (6) from primes (2×3), and an algorithm for a key part of Euclid's proof.

I illustrate the proof using Cuisenaire rods in figure 2.1. Imagine these rods represent lengths between 1 and 10, with the prime numbers being $\{2, 3, 5, 7\}$. The composite number 6 can be built using either 2 copies of 3 or 3 copies of 2 (or 6 copies of 1). Euclid's idea is elegant: take any finite collection of primes (say, A, B , and C), form a number DE that is their product, then add one unit to form $EF = DE + 1$.

This yields a fundamental incompatibility:

- **Case 1:** If EF is prime, it cannot equal any of A, B , or C (since it's larger than all of them), so it's a new prime outside the original list.
- **Case 2:** If EF is composite, it must have some prime factor G . But here's the crucial incompatibility: G cannot equal any of the original primes A, B , or C .

Why? If G were equal to one of them, then G would divide both DE (by construction) and EF (by assumption). But then G would have to divide their difference $EF - DE = 1$, which is impossible since no prime divides 1.

2.2.2 An Algorithmic Elaboration of Euclid to Incompatibility Semantics

I now approach the proof through incompatibility semantics. The key incompatibility is:

$$\{\text{The list of primes } \{A, B, C\} \text{ is complete}\} \in Inc$$

This statement belongs to the set of incoherent claims (Inc) because it generates an internal contradiction. Once we understand what prime numbers are, this claim defeats itself. The incoherence emerges through the following steps:

1. If the list $\{A, B, C\}$ is complete, then any newly constructed number must either be composite or equal to a prime already on the list.

2. The construction $N = A \times B \times C + 1$ yields a number that, whether prime or composite, forces the existence of a prime not on the list.
3. This contradicts the initial assumption of completeness, demonstrating that the claim to completeness is an element of an incoherence frame.

This example indicates that substituting incoherence for contradiction (among other substitutions) can be treated algorithmically. The number of historical proofs that admit such reconstruction is unknown, though many likely do. In this iteration, critical mathematics is not positioned as the adversary of prior work. Notably, the notion of an incoherence frame allows errors, misrecognitions, mistakes, and misconceptions to be **admitted** into the system without damaging its coherence. When an algorithm produces a paradoxical result, that paradox can be reconciled with the system by recognizing it as an element of an incoherence frame. The paradox is not a problem to be solved, but a new element to be incorporated.

This reconstruction of Euclid’s proof through incompatibility semantics casts history into the field of iteration. By translating Euclid’s geometric language about “lengths” and “measurement” into Brandom’s vocabulary of incompatibility relations, it becomes clear that the proof’s power comes from identifying a fundamental incoherence in the claim to completeness. The algorithmic substitution of “incoherence” for “contradiction” is not merely a terminological change but opens new possibilities for mathematical education and practice. Most significantly, incompatibility semantics allows errors, mistakes, and misconceptions to be admitted into mathematical systems without destroying their coherence – these apparent problems become elements to be incorporated rather than eliminated. This suggests that mathematical learning might proceed not through the avoidance of error but through the sophisticated recognition and integration of contradictory commitments, transforming potential barriers into resources for further development.

2.2.3 A Formal Reconstruction in Incompatibility Semantics

The conceptual reconstruction of Euclid’s proof can be taken a step further by formalizing it as a set of machine-executable rules within a Prolog-based incompatibility semantics engine. I felt compelled to try to formalize the proof above, and a few other claims in this book, out of existential fear. I figured that readers who followed along this far might be upset if the system I have been outlining was itself incoherent. So, I wanted to test some of those claims. This moves what was largely a subjective/normative set of validity claims into the realm of objective validity. However, as with all the claims in this book, readers should take

the formalisms with a grain of salt. I *tried* to ensure the system worked, but I had extensive help from various AI engines to finally cobble together enough code to ‘prove’ the theorems and claims. Still, from the Hermeneutic Calculator app to the Prolog code in the supplementary materials, readers can ‘see’ the claims in action. They work on some deeper level than just AI slop, even if they miss much of the nuance of the system as I imagine it could become. But that idea – what the system could become – required an appeal to the sort of expert that I am not. To lure those experts into thinking critically about critical mathematics, I felt I needed to provide enough formalism that those experts would understand in the sense of knowing how to act next from the text. You see, my ambition for the project is not to provide a finished system, but to provide a starting point for others to build on. Just as I have built on the work of others.

The most significant source that my formalisms draw on is Brandom’s *incompatibility semantics* (Brandom2008). The technical appendices in that work provide a symbolic logic that I transformed into Prolog. I added in the mathematical material inferences from my dissertation and a few key automata like the Highlander automaton. I also encoded the modal axioms for embodiment. I didn’t explicitly code in the concept of determinate negation, nor did I include $\mathcal{M}$ ’s concept of divided sets. Perhaps I should have, but I’m already a year past the deadline for completing this book and I figured that getting it out there in an *incomplete* form was a more honest representation of what I am trying to express anyway.

Those semantic rules form a normative material inferential framework. The Prolog engine, `more_machine_learner.pl`, draws on those rules to prove claims. The engine is not a theorem prover in the traditional sense. It does not take a set of axioms and try to prove a theorem from them. Instead, it takes a set of premises and tries to prove that the set is incoherent. This is a subtle but important difference. The engine is designed to model the inferential structure of Euclid’s proof, which hinges on demonstrating that the assumption of completeness leads to incoherence.

In the next chapter, I will discuss how the engine also ‘learns’ from observation. It only really has a counting automaton hard-coded into it. But it can access an anaphoric representation of an arithmetic strategy and then reconstruct the strategy based on its ability to count. That is, it writes a new automaton once it figures out what a user is saying when they write “ $5 + 7 = 5 + 5 + 2 = 10 + 2 = 12$.” In the next phase, I hope to ‘solve’ the unasked question “how many student-invented strategies are there in four-function arithmetic?” I took the problem of protein-folding as an analogy: the combinatorics of how four amino acids can be folded into a protein is more than can be brute-force computed. I know that the

combinatorics for how to combine all the digits between 1 and 1000 legally under four arithmetic functions is a bit too large for the human brain (mine anyway) to count. So, I am working on writing the engine to brute-force all those possibilities, analyze its output for possible efficiencies in base ten, and then output a set of strategies that are viable for human learners.

I don't know how such an 'answer' could be used by the community of math educators. However, as technology continues to colonize educational spaces, I figured that it would be better if the machines teaching our kids had access to *determinate* mathematical systems that were open to new ideas. The statistical inference patterns of large-language models have been correctly (in my opinion) derided as "random bullshit generators." Having that attribute makes them somewhat unsuited for educational spaces. I know this is contradicting my use of AI as I prepared the book! However, for the book, I have been in-the-loop, checking outputs and trying to rein-in some of the nonsense produced by LLMs. There's a difference between how I've been using AI as a tool and having teach my kids math.

Grounded Predicates and Material Inferences

At the foundation, the system needs to connect its symbolic reasoning to computational facts about numbers. This is achieved with *grounded predicates* that the engine can verify directly:

- `prime(N)`: Succeeds if N is a prime number.
- `composite(N)`: Succeeds if N is a composite number.
- `divides(D, N)`: Succeeds if D divides N .

Building on these, the core logic of Euclid's argument is captured in two key *material inferences*, which correspond to the crucial steps in his reasoning:

1. **M5 (Prime Divisor Incompatibility)**: Any prime factor G of the constructed number $N = (\prod L) + 1$ cannot be a member of the original list of primes L . This is the central incompatibility Euclid identifies. In our system, this is a conditional axiom:

```
proves([n(prime(G)), n(divides(G, N))] => [n(neg(member(G, L)))]), _ :-
    product_of_list(L, P), N is P + 1.
```

2. **M4 (Completeness Definition):** If there exists a prime G that is not on the list L , then the list L is not complete. This rule makes the meaning of “completeness” explicit in the system:

`proves([n(prime(G)), n(neg(member(G, L)))] => [n(neg(is_complete(L)))], _).`

Structural Rules and The Proof Algorithm

Astute readers will notice that I did *not* code in the whole text of Euclid’s *Elements*. Instead, I used some modern notions related to prime numbers and then enacted the procedure that Euclid uses to demonstrate incompleteness. The Prolog system models this procedure using *structural rules* that dynamically add new information to the set of premises, driving the proof forward.

1. **Construction:** Given the premise that a list of primes L is complete, the engine is entitled to perform Euclid’s construction, calculating $N = (\prod L) + 1$ and adding a new premise, `analyze_euclid_number(N, L)`, which triggers the next stage of reasoning.
2. **Case Analysis:** The `analyze_euclid_number` premise activates a case-split. The engine proves the goal first by assuming N is prime, and then proves it again by assuming N is composite. If the initial assumption of completeness leads to incoherence in *both* cases, the assumption is proven to be incoherent.
3. **Prime Factorization:** In the case where N is composite, another structural rule finds a prime factor G of N and adds the premises `prime(G)` and `divides(G, N)` to the knowledge base. This provides the necessary input for the material inferences (M4 and M5) to fire.

By combining these elements, the assertion that any finite list of primes L is complete, expressed as `incoherent([n(is_complete(L))])`, is proven. The engine begins with the single premise, applies the construction rule, proceeds through the case analysis, and uses forward-chaining over the material inferences to inexorably arrive at the conclusion that the list is *not* complete, rendering the initial premise incoherent. This formal elaboration transforms Euclid’s elegant proof into a running algorithm, demonstrating viability of incompatibility semantics.

2.3 Algorithmic Elaboration and Rules of Mathematical Inference

In the dissertation, I synthesized Lakoff and Núñez’s (Lakoff2000) embodied metaphors for mathematics with Brandom’s incompatibility semantics. At first, this was a challenge, but with familiarity the process began to feel algorithmic. I left the project incomplete, but I expect interested readers could read Lakoff and Núñez’s text, Brandom’s text, and then figure out how to discuss imaginary numbers based on their collective insights. I didn’t even complete multiplication and division! But the seed is planted – if others want to make it grow, they can water it.

Bounded Regions and Categories Initially, proprioceptive feelings define a boundary – comprising an *interior* (inside the body) and an *exterior* (outside the body). This does not include the concept of divided regions. The following rules govern the vocabulary associated with bounded regions:

1. If you are in a *bounded region*, you are not out of that *bounded region*.
2. If you are out of a *bounded region*, you are not in that *bounded region*.
3. If you are deep in a *bounded region*, you are far from being out of that *bounded region*.
4. If you are on the edge of a *bounded region*, you are close to being in that *bounded region*.

Figure

Figure 2.2: *Note.* This is an automaton that enacts a branched-schedule algorithm for articulating bounded regions. Note that it does not include divided regions.

Substitution Rules for Categories To transition from bounded regions to categories, apply the following substitutions:

1. Substitute “Categories” for “Bounded regions in space.”
2. Substitute “Category members” for “Objects inside the bounded regions.”

3. Substitute “A subcategory of a larger category” for “One bounded region inside another.”

This substitution allows the body-feeling to ground the notion of categories, maintaining a coherent vocabulary transition that supports the development of mathematical concepts. By algorithmically elaborating these substitution rules, the evolution from basic proprioceptive experiences to abstract mathematical categories can be made explicit, ensuring each step maintains logical consistency and normative correctness.

Educational Implications This approach mirrors pedagogical methods where complex mathematical vocabularies are built upon foundational concepts. Educators trace vocabulary transitions from simple to complex, allowing learners to internalize and apply new concepts seamlessly.

From Counting to Arithmetic Foundations

With numerical structures identified within a normative framework, arithmetic can be formalized by reinterpreting Lakoff & Núñez’s (**Lakoff2000**) work on embodied mathematics as material inferences within a deontic-normative framework. These material inferential rules are expressed through algorithms that encapsulate commitments and entitlements, enabling the foundation of arithmetic.

Challenges in First-Person Subjectivity Articulating mathematics from a first-person perspective introduces complexities, particularly when expressing concepts like *linearity* within a deontic-normative modality. Unlike the alethic modality, which deals with objective truth, the deontic-normative modality involves commitments and entitlements, necessitating a different approach to expressing mathematical laws.

Highlander Algorithm The *Highlander Algorithm* was already explicated in figure ?? . It embodies the principle that “only one of the following assertions can be committed to,;; implying the incompatibility of others. This algorithm is pivotal in enforcing exclusivity within mathematical commitments.

Equality-Iterator Algorithm The *Equality-Iterator Algorithm* formalizes the notion that different operations can yield the same result, adhering to the principle of equality of result.

Figure

Figure 2.3: *Note.* This algorithm expresses $A = B + C$ and $A = D + E$, where $B \neq D$ and $C \neq E$.

Material Inferences for Object Collections To establish arithmetic foundations, I explicate some material inferences that govern *object collections*. This enables the use of the vocabulary $V_{\text{object collection}}$.

Object Collection Vocabulary:

$V_{\text{object collection}} = \{\text{bigger, smaller, =, added to, results in, adding, result, subtracted from, subtract, zero}\}$

Material Inferences and Algorithms The following material inferences and algorithms formalize the operations on object collections:

1. **Linearity:**

$\{A \text{ and } B \text{ are object collections}\}^\dagger$

- $\{A \text{ is bigger than } B\} \models_I \{B \text{ is smaller than } A\} \rightarrow \neg\{A \text{ is smaller than } B\}$
- $\{B \text{ is bigger than } A\} \models_I \{A \text{ is smaller than } B\} \rightarrow \neg\{B \text{ is smaller than } A\}$
- $\{A = B\}$

2. **Closure:**

$\{A \text{ added to } B \text{ results in } D\} \models_I \{D \text{ is an object collection}\}$

3. **Commutativity:**

$\{\text{Adding } A \text{ to } B \text{ results in } C\} \models_I \{C = \text{Adding } B \text{ to } A\}$

4. **Associativity:**

$\{D \text{ results from adding } A \text{ to } (B + C)\} \models_I \{D = (A + B) + C\}$

5. **Transitivity:**

$\{A \text{ is bigger than } B \text{ and } B \text{ is bigger than } C\} \models_I \{A \text{ is bigger than } C\}$

6. Addition Structural Roles:

$\{A \text{ is an object collection in addition}\}^\dagger$

- $\{A \text{ is what is added to}\}$
- $\{A \text{ is what is added}\}$
- $\{A \text{ is the result of addition}\}$

7. Stuttering Inference:

$\{A\} \models_I \{A\}$

8. Unlimited Iteration for Addition:

$\{A \text{ is an object collection}\} \rightarrow \{A \text{ added to } B \text{ results in an object collection}\}$

9. Limited Iteration of Subtraction:

- $\{B \text{ is smaller than } A\} \rightarrow \{B \text{ subtracted from } A \text{ is an object collection}\}$
- $\{B \text{ is symmetrically intersubstitutable with } A\} \models_I \{\text{No objects left when } A \text{ is subtracted from } B\}$

10. Zero:

- $\{A \text{ is symmetrically intersubstitutable with } B\} \models_I \{\text{No objects left when } A \text{ is subtracted from } B\}$
- $\{A \text{ is an object collection with no objects}\} \rightarrow \{A \text{ is symmetrically intersubstitutable with zero}\}$

11. Sequential Operations:

- $\{B \text{ is smaller than } A\} \rightarrow \{B \text{ subtracted from } A \text{ added to } C \text{ results in an object collection}\}$
- $\{B \text{ added to } A \text{ results in a collection bigger than } C\} \rightarrow \{C \text{ subtracted from } (B + A) \text{ is an object collection}\}$

12. Equality of Result:

$\{A \text{ is an object collection}\} \models_I \{\text{Call Equality-Iterator Algorithm}\}$

13. Preservation of Equality:

$\{B \text{ and } C \text{ are symmetrically intersubstitutable}\} \models_I \{A \text{ added to } B \text{ is symmetrically intersubstitutable with } A \text{ added to } C\}$

Symbol Legend

Table 2.1: *Note.* **Legend for Symbols Used**

Symbol	Substituted-for
$\neg$	Precludes entitlement to.../is incompatible with
$\rightarrow$	Entitles/has as an inferential consequent...
$\models_I$	Entitles/incompatibility-entails ...
$\dagger$	Highlander algorithm for “entitles only one of the following commitments”
$\rightarrow_{SI}$	Simple Material Substitution Inferential Commitment
$A = B$	A is symmetrically intersubstitutable with B

Object Collections to Arithmetic By defining these material inferences and algorithms, the foundational operations on *object collections* that underpin arithmetic over numbers are established as embodied normative practices. This process ensures that arithmetic operations are grounded in normative, deontic-inferential structures rather than purely alethic assertions.

This systematic elaboration from embodied metaphors to formal arithmetic demonstrates how complex mathematical concepts emerge through algorithmic chains of material inference. By beginning with basic proprioceptive experiences of bounded regions and algorithmically substituting abstract categories, it becomes evident how mathematical vocabulary develops through normatively regulated transformations rather than arbitrary formal definitions. The detailed specification of material inferences for object collections – including linearity, closure, commutativity, and associativity – shows how the familiar laws of arithmetic arise from implicit commitments embedded in practices of counting and collecting. Most importantly, this approach grounds mathematical operations in deontic-normative structures (what agents ought to be committed to given their other commitments) rather than alethic assertions about objective mathematical facts. This reveals arithmetic not as a description of abstract objects but as an articulation of the implicit inferential norms that govern mathematical practices, preparing readers to understand the limits of such systematic elaboration and the need for more dialectical forms of conceptual development.

2.4 The Limitations of Algorithmic Elaboration: Dialectical Moments

Algorithmic elaboration is a powerful tool for understanding the evolution of mathematical concepts. However, it has a cousin that is not so easily tamed: dialectical moments. These moments are characterized by a clash of incompatible commitments that cannot be resolved through simple elaboration. They require a more nuanced approach, often involving a shift in perspective or a re-evaluation of the foundational assumptions. While I cannot further formalize the dialectic, another avenue for exploration is Brandom's pragmatic expressive bootstrapping. In some respects, it is an extension of diagonalization to the realm of logical expressivism.

2.4.1 Brandom's Pragmatic Expressive Bootstrapping

The concept of *pragmatic expressive bootstrapping*, provides another avenue for understanding the dialectical movement of mathematical and conceptual history (Brandom2008). It describes the process by which new, more expressively powerful vocabularies emerge from implicit, embodied practices already mastered by practitioners. This is the mechanism by which a conceptual system, in a sense, pulls itself up by its own bootstraps, making explicit what was previously only latent within its own doing.

This process mirrors the structure of *diagonalization* explored in the Bridge chapter. In diagonalization, a system (like a list of numbers or a set of axioms) is used to construct a new element that is demonstrably part of the system's potential but lies outside its initial, explicit enumeration. Similarly, in pragmatic expressive bootstrapping, a set of existing practices – embodied “knowing-how” – contains the implicit resources to make explicit a new vocabulary that transcends the expressive power of the language used to describe those practices. The key components of this process are:

- *Pragmatism*: The foundation of the process is in discursive *practices* and *abilities*. It begins not with abstract rules or meanings, but with the concrete, embodied doings that constitute engagement with the world and each other. This aligns with the theme that the lived, felt experience of mathematics is prior to its formalization.
- *Expressivism*: The function of the new, bootstrapped vocabulary is *expressive*. It does not invent new content out of thin air, but rather makes explicit

the inferential norms and commitments that were already implicit in shared practices. It is the process of language recollecting the structure of its own use.

Brandom formalizes this relationship through his Meaning-Use Analysis, which distinguishes between a vocabulary being sufficient to *specify* a practice (*VP-sufficiency*) and a practice being sufficient to *deploy* a vocabulary (*PV-sufficiency*). Pragmatic expressive bootstrapping occurs when a vocabulary (V') is VP-sufficient to specify practices (P) that are, in turn, PV-sufficient to deploy another, more expressively powerful vocabulary (V) (**Brandom2008**). The “bootstrapping” happens when the specifying vocabulary (V') is weaker than the deployed vocabulary (V).

A prime example is the emergence of logical vocabulary. The practices of making and assessing ordinary material inferences (e.g., “Pokey is a dog, so Pokey is a mammal”) are implicit in any autonomous discursive practice. These practices themselves do not require logical vocabulary. However, these doings are sufficient to allow for the introduction of logical vocabulary, such as the conditional (“if...then...”), which then makes those material inferential commitments explicit. The logical vocabulary is thus *elaborated from* and is *explicative of* the underlying practices. Brandom calls this an “LX” vocabulary (**Brandom2008**).

This is significant for a critical mathematics. Pragmatic expressive bootstrapping reveals that the abstract structures of mathematics and logic are not alien, Platonic forms imposed upon the practitioner. Instead, they are the result of discursive practices recollecting themselves, making their own implicit structures explicit. This is an emancipatory insight. It reframes mathematical knowledge not as a static body of truths to be passively received, but as a dynamic, ongoing process of conceptual creation. It shows how learners, through their implicit practices, generate the tools that allow them to transcend their current understandings, turning the implicit ground of embodied rationality (see ??) into the explicit, shared world of mathematical reason. This is the formal-pragmatic counterpart to the dialectical movement from the subjective and embodied to the objective and universal.

Pragmatic expressive bootstrapping provides the theoretical framework for understanding how mathematical systems achieve genuine transcendence of their apparent limits. Like the diagonalization explored in the Bridge chapter, bootstrapping reveals how conceptual systems contain within themselves the resources for their own development – not through external imposition but through the systematic explication of their implicit normative structure. This process connects

directly to the pronoun thesis of Chapter 7: just as numerals function as anaphoric recollections (recall ??) of the enabling conditions of thought, logical vocabulary emerges as the explicit recollection of the material inferential practices we already implicitly master. The significance for mathematical education is profound: rather than viewing abstract mathematical concepts as mysterious entities to be passively received, students can understand them as sophisticated articulations of the practical abilities they already possess. This transforms mathematics from an alien imposition into a process of self-recognition and self-development, revealing how the movement from embodied practice to formal reasoning is not a loss of humanity but a sophisticated form of self-expression.

2.5 Formalizing Bootstrapping Procedures

Algorithmic elaboration is powerful for making the implicit rules of a given practice explicit. However, conceptual history also contains dialectical moments where the system of rules itself must be transcended. These moments occur when a system of normative commitments confronts an observation that it cannot account for – an observation that is *incoherent* under its current structure. Resolving this requires **pragmatic expressive bootstrapping**: the process of generating a new, more expressively powerful conceptual system from the resources implicit in the old one (**Brandom2008**).

My work has not only theorized this process but implemented a model of it. The `more_machine_learner.pl` script provides a computational model of a system engaging in normative critique and bootstrapping to overcome its own expressive limitations. Let's examine its operation, which unfolds in four distinct phases.

Phase 1: The Initial Normative Stance

The system begins with a set of norms equivalent to arithmetic over what Lakoff & Núñez call “object collections” (**Lakoff2000**), which corresponds to the domain of Natural Numbers ($\mathbb{N}$). Within this framework, subtraction is limited. As defined by the material inferences in section 2.3, one can only subtract a smaller collection from a larger one. This limitation is formalized as an incoherence axiom in the system's logic (domain $\mathbb{N}$ ‘n’):

```
% An assertion that subtraction resulting in a non-natural number exists
% is incoherent in the domain of Natural Numbers.
```

```
is_incoherent(X) :-
    member(n(obj_coll(minus(A,B,_))), X),
    current_domain(n), number(A), number(B), A < B.
```

This rule explicitly defines the boundaries of the current conceptual scheme: the idea of $3 - 5$ is, for this system, nonsensical.

Phase 2: The Normative Critique

The dialectical moment is initiated when the system is presented with an external observation that violates its norms. For example, it observes the fact of `minus(3, 5, -2)`. The system's first step is to assess this observation against its current commitments:

```
?- critique_and_bootstrap(minus(3, 5, -2)).
```

```
-- Normative Critique Initiated --
Observation: minus(3, 5) = -2
Current Domain: n
```

Observation is INCOHERENT with current norms. Entering critique phase...

The system correctly judges the observation to be incoherent. Rather than simply rejecting it as “false,” the system recognizes a fundamental clash. It then enters a critique phase to diagnose the precise nature of this incoherence. It identifies that the problem is not with the specific numbers, but with the very structure of subtraction in its current domain:

```
Identified Incompatibility: incompatibility(limited_subtraction, domain(n))
```

Phase 3: The Normative Shift (Bootstrapping)

The identification of the specific limitation allows the system to propose a solution. It suggests a *normative shift* to a more expressively powerful domain: the Integers ($\mathbb{Z}$, represented as domain ‘z’), where the limitation on subtraction is removed.

```
Proposed Normative Shift: Change domain to z
Adopting new norms...
```

This is the “bootstrapping” move. The system uses the failure of its old vocabulary (‘V’: arithmetic over $\mathbb{N}$) to specify a practice (‘P’: subtraction) that is sufficient to deploy a new, more powerful vocabulary (‘V’: arithmetic over $\mathbb{Z}$). The new domain is not arbitrary; it is precisely the domain required to make sense of the practice that broke the old one.

Phase 4: Coherence in the Elaborated System

After adopting the new normative framework by changing its domain to ‘z’, the system re-assesses the original observation.

Bootstrapping complete. Observation is now coherent.

-- Normative Critique Concluded --

The operation $\text{minus}(3, 5, -2)$ is now provably coherent. The paradox has been resolved not by dismissing the observation, but by transforming the conceptual framework. The system has pulled itself up by its own bootstraps, using an incoherence generated within its own practices to explicate and adopt a more sophisticated set of norms. This computational example demonstrates that the dialectical movement from one conceptual stage to the next can be understood as a rational, rule-governed process of critique and repair, turning expressive limitations into engines for conceptual growth.

2.5.1 Reflecting on the Materiality of Mathematical Inferences

The formalization of pragmatic expressive bootstrapping in the `more_machine_learner.pl` script illustrates how dialectical moments in conceptual history can be understood as rational processes. The system’s ability to recognize its own limitations, critique them, and adopt a more powerful framework mirrors the historical development of mathematical concepts. Just as ancient mathematicians grappled with the limitations of their number systems, leading to the invention of negative numbers and beyond, this model shows how such transitions can be systematically articulated.

Moreover, the formalism may help convince those who are convinced that mathematical reasoning *should* be monotonic that it isn’t. That does not mean that pockets of monotonic reasoning are impossible, like the ‘small sodium moons’ I poetically described in *Breath and Kindling*. The rule that $a - b$ only works when $b \leq a$ is good until it isn’t. Like Brandom’s dry, well-made match that won’t light in a strong electromagnetic field (Brandom2000), the subtraction rule won’t work in the context of the integers.

Materiality in mathematics proper need not show up as frequently as it shows up in, say, an elementary classroom. Instead, it may show up in the ‘grown-up’ world navigated by mathematicians as the bridge between formal systems. Multiplication commutes, unless you’re working in a non-Abelian group, for example. My claim that mathematics is material provides an explanation for the fluidity of reasoning that mathematicians demonstrate when they transition between thinking in one formal system to another.

2.6 A Next Step: Hylomorphism and Poetic Computation

The philosophical framework guiding this project draws on a revitalized concept of hylomorphism – the Aristotelian doctrine that entities are a composite of form (*morphē*) and matter (*hylē*) (inwood1992hegel). In the bimodal hylomorphic conceptual realism Robert Brandom attributes to Hegel, this idea is applied to concepts themselves (Brandom:2019aa). A single conceptual content – the “matter” – can exist in two distinct forms: an **objective form** governed by alethic modalities of necessity and impossibility, and a **subjective/normative form** governed by deontic modalities of commitment and entitlement (Brandom:2019aa). This section proposes a next step for our computational model: to leverage the unique features of Prolog to explicitly model this hylomorphic structure, thereby opening a path toward a formal model of poetic ambiguity and conceptual creativity.

2.6.1 Homoiconicity as the Key

The mechanism that makes this modeling possible is Prolog’s **homoiconicity** – the principle that code and data are made of the same “stuff.” In Prolog, a term such as `plus(2, 3, 5)` is not merely a string of characters; it is a structured piece of data that can be passed as an argument, inspected, and decomposed. Simultaneously, it can be treated as code – a goal to be proven by the inference engine. This dual nature of terms is the perfect computational analogue for the matter/form distinction of hylomorphism.

Our existing system already intuits this. When the engine reasons about `o(plus(2, 3, 5))`, it is treating the underlying term `plus(2, 3, 5)` as the conceptual “matter.” The `o()` prefix acts as the “form,” marking the content as having an objective, alethic modality. The same matter can be given a different form, as in `n(plus(2,`

3, 5)), which marks it as a normative claim. The logic engine can reason about the term as a whole, or it can “look inside” the form to reason about the matter it contains.

The next step is to make this “hylomorphic shift” an explicit, rule-governed operation within the Prolog engine. We can introduce new ****structural rules**** that allow the system to re-interpret a concept by formally changing its modal form. For instance, we could implement a rule that models the Hegelian dictum “what is rational is actual” (**taylor2015hegel**). This rule would state that if a proposition P is found to be objectively true, the system is entitled to a normative commitment to P :

```
% Hylomorphic Shift Rule: From Objective to Normative
proves([o(P)] => [n(P)], _).
```

This rule allows the system to move from recognizing a fact about the world to adopting a norm for belief. The “creative moment” would be the system’s application of this rule, which reframes the meaning of P from a description of reality to a prescription for thought. It treats the same conceptual “matter” in a new way.

This opens the door to modeling more complex dynamics. What if the system could learn when to apply such a shift? It could, for instance, encounter a set of normative commitments that are incoherent, and resolve the incoherence by searching for an objective state of affairs that could ground a new, coherent set of norms. This process – where the demands of subjective consistency (the “for consciousness” form) drive an inquiry into objective reality (the “in itself” form) – is a computational microcosm of the process of gaining knowledge. By formalizing the hylomorphic ambiguity of concepts, we can begin to model not just the logic of static systems, but the creative, self-correcting dynamic of thought itself.

2.7 Conclusion

This chapter has sought to demonstrate that the history of mathematics, and by extension conceptual history more broadly, can be understood as a rational process of algorithmic elaboration. By moving from the implicit practices of counting, measuring, and inferring, the explicit, formal systems of arithmetic and logic are elaborated. The reconstruction of Euclid’s proof using incompatibility semantics shows how even ancient mathematical insights can be understood as the formal articulation of underlying inferential commitments. The process reveals that a

claim to completeness is self-defeating, not merely false, thereby opening the door to the *infinite*.

However, the journey of knowledge is not a simple, linear unfolding. The limitations of algorithmic elaboration point to the necessity of dialectical ruptures that require a more profound form of conceptual change. Brandom's pragmatic expressive bootstrapping provides a model for these moments. It shows how a system of practices can, in a move analogous to diagonalization, generate a new, more expressively powerful vocabulary that makes its own implicit normative structure explicit. This is not magic, but language unfolding itself. It reframes mathematical knowledge not as a static collection of truths, but as a dynamic, self-correcting, and emancipatory process: a constant becoming.

By understanding the rules implicitly followed, it becomes possible to make them explicit, critique them, transcend them, or identify with them.

Chapter 3

Operation

Abstract

This chapter explores the nature of mathematical operations, arguing that they are not simply formal manipulations of symbols but expressions of embodied, normatively regulated practices. The chapter develops a framework called “critical arithmetic,” a pre-formal system capturing the dynamism of everyday arithmetic, including errors. Through the lens of multiplication, the chapter demonstrates how operations emerge from a triad of perspectives: embodied practices, inferential rules, and formal structures. The analysis begins with the embodied basis of arithmetic, drawing on examples like the “Coordinating Two Counts by Ones” (C2C) strategy, illustrating how children invent strategies grounded in physical interaction. The chapter then articulates how these subjective experiences transform into shared, rule-governed practices through a normative framework based on Brandom’s incompatibility semantics. Finally, it demonstrates how these practices give rise to stable, objective procedures, modeled as formal automata. By connecting Lakoff and Núñez’s grounding metaphors to this normative framework, the chapter explains how formal arithmetic structures emerge from lived experience. This focus on critical arithmetic challenges traditional views of operations as fixed procedures and promotes a more dynamic and creative understanding of mathematical meaning-making. The reader will gain a deeper understanding of the complex interplay between embodiment, social norms, and formal systems in the development of mathematical understanding.

3.1 Introduction: From Numbers to Operations

The preceding chapters reconstructed the nature of number itself.

I began with a student's question asked in a moment of profound grief and frustration: "Mr. Savich, what even is two?"

That moment led away from the conventional view of numbers as abstract, pre-existing objects and toward an account of them as first-person recollective pronouns.

Numerals, symbolized by the null representation $\emptyset$ (see ??), emerge from the dynamic interplay of linguistic self-reference, anaphorically recollecting the enabling conditions of thought – the transcendental "I think."

Having explored what numbers *are* – reflections of dynamic, self-recollecting human being – this chapter now turns to the equally fundamental question of what it means to *do* things with them.

Conventional accounts treat mathematical operations as formal manipulations of abstract symbols, governed by axioms that are either arbitrary conventions or self-evident truths. In this view, addition, subtraction, multiplication, and division are processes applied to numbers, with rules to be memorized and followed.

I propose a different understanding, continuous with the emancipatory project of this book: mathematical operations are not arbitrary conventions but are themselves expressions of embodied, normatively regulated practices. The "laws" governing the use of numbers are best understood not as transcendent truths, but as algorithms or inference chains – normatively regulated doings that emerge from embodied engagement with the world and with others.

This chapter articulates inference rules for what I call a "critical arithmetic" – an open-ended, pre-formal system that captures the kind of arithmetic used in everyday life – errors and all.

To unpack this, the chapter follows the triadic structure of validity and accompanying modal logics developed throughout the inquiry, examining operation from three integrated subject positions:

1. **The Subjective Validity of Embodied Practices.** Begin with the first-person, lived experience of mathematical activity. Drawing on the insights of embodied cognition (**Lakoff2000**), arithmetic operations are grounded in basic, physical interactions with the world. This is the material, felt ground of mathematical meaning.

However, while Lakoff and Núñez (**Lakoff2000**) provide a beautiful and rich account of arithmetic, they do not frame their work from within em-

bodied experience. Instead, they take the “god’s eye” view of cognitive science. I therefore transpose their metaphors into the embodied modal logic developed in Chapter 1.

2. **The Normative Validity of Inferential Rules.** The analysis then moves to the second-person, intersubjective space of giving and asking for reasons. Here, I employ Brandom’s incompatibility semantics to show how the “rules” of arithmetic are not imposed from without but are made explicit by structuring embodied practices through commitments and the exclusion of error (**Brandom2008**). This normative framework governs the “goodness” of an operation. I again draw on the metaphors of Lakoff and Núñez, but this time (as in my dissertation), I show how to transpose the metaphors into a system for excluding errors based on ???. “The Exercise,” and the modal logic that reflects it, is not particularly concerned with errors or exclusions except insofar as they arise by acting on temptation. For this reason, an additional logic provides the material-exclusionary rules that Brandom articulates.

The errors of interest can be gleaned from the phylogenetic research of mathematics education. Much of that work concerns various kinds of errors, misconceptions, and perturbations that arise in learning mathematics. In my dissertation, I used data from Erik Jacobson’s work that I assisted with. For brevity, I leave it to the reader to explore the literature on errors in math education, under the assumption that many will recognize the kinds of errors that can arise. I certainly do.

3. **The Objective Validity of Formal Structures.** Finally, these normatively governed practices give rise to stable, repeatable, and objective procedures that can be modeled as formal automata. To reflect the I–Thou in what are usually considered lifeless structures, I again draw on the phylogenetic research of mathematics education. Specifically, I formalize the invented strategies of children that Carpenter *et al.* (**Carpenter1999**) codified in their video series for Cognitively Guided Instruction (CGI).¹ The absolute alterity of the Other is central to the radical constructivist research base most

¹Sebastian Rödl’s *Self-Consciousness and Objectivity* (**Rödl:2018aa**) argues for understanding Hegel’s absolute idealism as the unity of self-consciousness and objectivity. I lack the space to reconstruct that argument, but it should be familiar to many Hegelian scholars. Poetically, in absolute idealism, the subjective and objective poles of experience meet. I see no issue with subsuming the embodied modality into the alethic modality, but leave that issue for future exploration.

familiar to me due to proximity to that unfolding project. I provide about twenty-five automata that cover the four basic arithmetic operations in the Hermeneutic Calculator app ([savich_hermeneutic_nodate](#)), to represent the third-person objectified pole of this triadic monism.

This introduction establishes the chapter’s completion of the critical mathematics framework through its focus on mathematical operations as embodied, normatively regulated practices. Having explored numbers themselves as first-person pronouns recollecting the enabling conditions of thought, we now turn to what it means to do things with numbers – revealing operations not as formal manipulations of abstract objects but as expressions of our embodied engagement with the world. The triadic approach integrates the subjective experience of mathematical doing, the normative rules that structure these practices, and the objective forms that emerge from their systematization. This approach promises to show how the formal structures of arithmetic emerge from lived experience rather than being imposed upon it, completing the emancipatory vision of mathematics education that runs throughout the book. The focus on “critical arithmetic” as an open-ended, error-inclusive system challenges traditional approaches that treat operations as fixed procedures, instead revealing them as dynamic practices capable of creative development and meaningful error integration.

This chapter, in essence, provides a recipe for transposing the embodied metaphors of Lakoff and Núñez into the material inferential framework of Brandom. It aims to show how the formal emerges from the felt, how the abstract emerges from the concrete, and how operations, at their core, are a meaningful expressions of our being in the world.

3.2 Deconstructing Operations: The Case of Multiplication

Mathematical operations are often taught as monolithic, fixed procedures. A deeper understanding, however, reveals them as complex conceptual activities that can be approached and elaborated in a multitude of ways. This chapter uses multiplication as a running example to deconstruct the nature of “operation” and to demonstrate the “recipe” for moving from embodied practice to formal structure.

3.2.1 The Embodied Ground: Subjective Validity

The journey of any operation begins not with a symbol, but with a doing. Children, before they are ever taught a formal multiplication algorithm, invent their own strategies for solving problems involving multiplicative reasoning (**Carpenter1999**). These strategies are not random; they are grounded in their embodied experience of the world.

Consider one of the most intuitive of these strategies, a practice that emerges directly from the fusion of counting and grouping: *Coordinating Two Counts by Ones (C2C)*.

- **Objective:** To find the total number of items arranged in a certain number of equal-sized groups (e.g., 3 groups of 4).
- **Method:** For the first group, count the items by ones: “one, two, three, four.” For the second group, continue counting by ones: “five, six, seven, eight.” For the third group, continue the count: “nine, ten, eleven, twelve.” All the while, a second, implicit count tracks the number of groups that have been processed.

This is a lived, felt process. It is a rhythmic activity – a coordination of gesture, vocalization, and attention – grounded in the basic human capacities for grouping objects, maintaining a sequence, and tracking multiple streams of information. This represents the *subjective* pole of the operation. It is a private, embodied practice that makes intuitive sense to the person performing it. It is a form of knowing that is inseparable from the physical act of doing. It is a claim to subjective validity, an assertion of “this is how it feels right to me to do this.” This is the primordial material from which all formal mathematics is built.

3.2.2 The Recipe: From Embodiment to Norms

How do we move from this private, embodied feeling to a shared, communicable, and rule-governed practice? A private feeling is not yet a mathematical operation. For it to become one, it must enter the intersubjective space of giving and asking for reasons. It must become subject to normative assessment. This requires a recipe for making the implicit norms of the practice explicit.

1. **Start with the Embodied Practice:** We begin with an intuitive strategy like C2C, grounded in the subjective validity of embodied experience.

2. **Articulate Material Inferences:** We then re-describe this practice as a set of material inferential commitments using the framework of incompatibility semantics (**Brandom2008**).

The “goodness” of an operation is not an abstract, pre-existing truth, but a normative status conferred upon a practice within a recognitive community. A calculation is deemed “correct” if it follows from a set of commitments that are coherent. Error, in this view, is that which is *incompatible* with the established commitments of the practice. The operation $3 \times 4 = 12$ can be articulated not as a timeless truth, but as a material inferential commitment:

$$\{3 \text{ groups of } 4\} \models_I \{12\}$$

Using the definition of incompatibility entailment, this means that everything incompatible with the set of commitments associated with the result ‘12’ (e.g., asserting the result is 11 or 13) must also be incompatible with the initial premise ‘3 groups of 4’. By systematically excluding what is incoherent, we define the boundaries of the coherent practice. This network of material inferences, such as those articulated for object collections in my dissertation (**savich2022**), constitutes the *normative* structure of the operation. The subjective feeling of “rightness” is thus transformed into a public, shared standard of correctness.

This analysis of multiplication reveals the crucial transition from private, embodied experience to public, normatively regulated practice. The child’s intuitive strategy of Coordinating Two Counts by Ones represents the subjective pole – a rhythmic, felt activity grounded in basic human capacities for grouping and sequential tracking. But for this embodied practice to become mathematical, it must enter the intersubjective space of giving and asking for reasons, where material inferential commitments make explicit what was implicit in the lived experience. The transformation of the feeling “this seems right to me” into the public standard “this is correct according to our shared commitments” exemplifies how mathematics emerges from embodied practice rather than being imposed upon it. This process respects both the creativity of individual mathematical thinking and the necessity of shared standards, showing how subjective validity can develop into normative coherence without losing its grounding in lived experience.

3.2.3 The Phenomenological Ground of Operation

I now transpose the four foundational grounding metaphors for arithmetic into the modal logic developed in Chapter ???. Each will be revealed as a particular

rhythm of temporal compression and decompression, a dialectical movement between Reification ($[T]$) and Sublation ($[LG]$).

Arithmetic as Object Collection

This is the most fundamental metaphor, grounding our understanding of number and addition in the physical act of gathering objects into groups. In our modal logic, this is not a simple mapping but a complex phenomenological performance.

- **The Objects (The Numbers Themselves):** To perceive a discrete object is an act of **temporal compression**. It is a movement from the primordial, undifferentiated state of Being (S) into a Desire (D) to parse the perceptual field, a temptation that is then fixed through an act of **Reification and Commitment** ($[T]$). An object – and thus a number like “four” representing a collection of four such objects – is a state of **Picture Thinking** (T). It is a normative commitment to a particular, finite differentiation of experience.

$$S \rightarrow \langle \text{Awareness} \rangle A(S) \rightarrow \langle \text{Temptation} \rangle D \rightarrow [T]\phi$$

where ϕ is the reified object or number.

- **The Act of Collection (Grouping):** This is an act of **temporal decomposition**. To form a “collection,” one must perform an act of **Sublation and De-Commitment** ($[LG]$) on the individual identities of the objects. One must “let go” of the commitment to their distinctness, allowing them to fuse into a new, undifferentiated unity. This is a move from multiple committed states ($T_1, T_2, \dots$) back towards a unified (though localized) state of Intensified Sensation (S').
- **Recollecting the Collection:** This is the final act of **temporal compression**. Having de-committed from the individual objects to form a unified perceptual field, one now undertakes a new act of Reification ($[T]$) on that unity. The *collection itself* becomes a new object, a new thought-form (T).

Under this transposition, **addition** (e.g., 3 groups + 4 groups) is revealed as a dialectical rhythm of commitment, de-commitment, and re-commitment. One begins with two reified thought-objects (T_3, T_4), lets go of their distinctness ($[LG]$) to enter a state of potential (S'), and then reifies this new totality ($[T]$) as a new determinate object (T_7). **Subtraction** is the inverse process: an analytic commitment

that differentiates a part from a pre-existing whole. And **Zero** is the primordial state S itself – the state of total decompression where no commitments ($[T]$) to determinate objects have yet been made.

Arithmetic as Object Construction

This metaphor adds the idea of numbers as wholes made up of parts that fit together in a specific structure. The transposition is similar to Object Collection, but the act of Reification ($[T]$) is more complex. The commitment is not to an undifferentiated totality, but to the *internal relations* between the parts. The final thought-object (T) is a commitment to a determinate structure. The act of “splitting” a whole into fractions is thus a profound act of structural Reification, where the commitment is not just to a part, but to a *principle of division* that governs the whole.

The Measuring Stick Metaphor

This metaphor, which conceptualizes numbers as physical segments and addition as placing them end-to-end, directly engages Axiom 4 (Intensification through Repetition). The “unit” is the foundational Reification ($[T]$) that establishes a normative standard. The act of measurement is then a phenomenological rhythm:

1. Commit to the first segment’s placement: $[T]$.
2. Let go of its specific location to move to the next: $[LG]$.
3. Commit to the second segment’s placement: $[T]$.

This is precisely the dialectical movement of Axiom 4: $(S_n \wedge D) \rightarrow \langle LG \rangle S_{n+1}$. Each measurement is a cycle of compression and decompression that “accumulates” and “intensifies” the result. The sequence of natural numbers, the backbone of arithmetic, emerges from this rhythm of doing and letting go.

Arithmetic as Motion Along a Path

Here, numbers are locations and addition is movement. The **Origin** is the state of Being, S , before any committed act of displacement. A **number** is a reified state (T) that is the result of a process of motion. This framework provides a powerful way to understand the emergence of **negative numbers**.

A positive number is a commitment to movement in a certain direction away from the origin. Multiplication by -1 , which corresponds to a 180-degree rotation,

3.2. DECONSTRUCTING OPERATIONS: THE CASE OF MULTIPLICATION⁴⁹

can be conceptualized as a profound **Sublation** ($[LG]$) of this initial directional commitment. This act of “letting go” of the directionality does not return one to ignorance, but to an intensified state (S') that now contains the integrated potential for movement in *either* direction. A new commitment ($[T']$) can then be made in the opposite direction. Thus, multiplication by -1 is a dialectical movement of $[T] \rightarrow [LG] \rightarrow S' \rightarrow [T']$. This explains the feeling of “flipping” to the other side of the number line not as a formal rule, but as an internal, cognitive act of de-commitment and re-commitment.

By transposing these grounding metaphors, we see that the fundamental structures of arithmetic – objecthood, grouping, iteration, and displacement – are not cold, abstract, third-person concepts. They are, in our modal logic, the rhythms of conscious experience: the constant dialectical pulse of Reification and Sublation, of Desire and Letting Go, of temporal compression and decompression. The “sound of time” is the rhythm section that accompanies the unfolding of mathematics.

This transposition of Lakoff and Núñez’s embodied metaphors into first-person modal logic reveals arithmetic operations as dynamic temporal processes rather than static conceptual mappings. Each fundamental metaphor – Object Collection, Object Construction, Measuring Stick, and Motion Along a Path – becomes a specific rhythm of consciousness, a dialectical movement between commitment and letting go that manifests as mathematical thinking. The analysis shows that addition, subtraction, multiplication, and division are not external procedures applied to numbers but internal movements of conscious experience made explicit through mathematical notation. Most significantly, this framework explains the feeling of mathematical understanding: when operations “make sense,” it is because they align with the deep rhythmic structures of embodied consciousness. This grounding provides the material basis for the normative rules that follow, showing how the “laws” of arithmetic emerge from the temporal structure of human experience rather than being imposed as arbitrary conventions or discovered as eternal truths.

3.2.4 The Embodied Ground of Norms: Transposing Lakoff and Núñez

The recipe for moving from subjective feeling to normative rule requires a deeper look at the ground from which these practices emerge.

These third-person observations can, therefore, be transposed into a second-

person, deontic-normative key.

The most basic grounding metaphor, **Arithmetic as Object Collection**, is used to demonstrate this transposition.

The following are not timeless, objective truths, but are material inferential commitments that constitute the coherent practice of reasoning about collections. To fail to endorse them is not to be wrong about the world, but to be incompetent in the shared language game of basic arithmetic.

1. **Linearity:** Lakoff and Núñez observe that our experience with physical collections grounds the concept of linear ordering. Any two distinct collections are comparable in size. This embodied reality is made explicit as a normative commitment, enforced by the Highlander Algorithm, which ensures that for any two collections, one and only one of three possible commitments can be undertaken.

$$\{A \text{ and } B \text{ are object collections}\}^\dagger$$

- $\{A \text{ is bigger than } B\} \models_I \{B \text{ is smaller than } A\}$
- $\{B \text{ is bigger than } A\} \models_I \{A \text{ is smaller than } B\}$
- $\{A = B\}$

To assert that A is both bigger than and equal to B is to make an incoherent move, to undertake incompatible commitments. The normativity here is not in the world, but in the practice of speaking coherently about it.

2. **Commutativity:** The physical act of combining a collection of three things with a collection of four things yields the same result as combining four with three. While the embodied *actions* are different in their temporal sequence, the practice of counting collections commits us to the equivalence of their results. This becomes a rule for the correct use of the vocabulary of “adding.”

$$\{\text{Adding } A \text{ to } B \text{ results in } C\} \models_I \{C = \text{Adding } B \text{ to } A\}$$

This expresses a normative commitment to the symmetrical intersubstitutability of the results of these two distinct temporal processes. A speaker who claims that the order matters is not describing a feature of the universe, but is failing to follow the established norms of the practice of arithmetic.

3.2. DECONSTRUCTING OPERATIONS: THE CASE OF MULTIPLICATION 51

3. **Associativity:** Similarly, our embodied experience of grouping collections is associative. If we add a collection A to the result of combining B and C , we are committed to acknowledging that this is the same result as adding the combination of A and B to C .

$$\{D \text{ results from adding } A \text{ to } (B + C)\} \models_I \{D = (A + B) + C\}$$

This is not an axiom handed down from on high, but a structural feature of our embodied practice of grouping, made explicit as a normative rule of inference. It ensures that the way we bracket our sequential operations does not lead to incompatible results, thereby preserving the coherence of the practice.

4. **Closure:** The practice of adding collections always results in another collection. This is not a metaphysical guarantee, but a fundamental commitment that defines the boundaries of the practice itself.

$$\{A \text{ added to } B \text{ results in } D\} \models_I \{D \text{ is an object collection}\}$$

This rule ensures that the activity of addition does not force us out of the language game of reasoning about object collections. It is a commitment that makes the practice of arithmetic stable and self-contained.

This transposition reveals the profound connection between our analysis and that of Lakoff and Núñez. Their embodied metaphors describe the *material* basis of mathematical concepts – the stable patterns of our sensory-motor experience. Our normative-inferential framework shows how these material patterns are transformed into *rules* for reasoning – the shared, public norms that govern the correct application of these concepts. This is the bridge between the felt, subjective sense of a practice and its formal, objective articulation. It is how the “sound of time,” in its rhythmic manifestation as embodied action, becomes the structured logic of mathematical operation.

This systematic transposition demonstrates how the embodied basis of mathematical understanding can be made explicit as normative commitments without losing its grounding in lived experience. The properties of arithmetic – linearity, commutativity, associativity, and closure – are revealed not as abstract axioms but as structural features of embodied practice made explicit through incompatibility semantics. By showing that mathematical “laws” are actually normative commitments that structure coherent reasoning rather than objective facts about

mathematical objects, this analysis preserves both the felt sense of mathematical meaning and the rigor of formal reasoning. The framework suggests that mathematical education should help students recognize these normative commitments as elaborations of their own embodied practices rather than external impositions. This completes the bridge from the subjective experience of mathematical doing to the shared, intersubjective space of mathematical reasoning, preparing for the final step toward formal, objective structures.

3.3 The Subtlety of Sameness: Commutativity and Structural Transformation

With a basic operation grounded, the next step is to explore its properties.

The commutative property of multiplication ($a \times b = b \times a$) is often presented as a simple, self-evident fact. However, from the embodied, conceptual standpoint developed here, it involves a profound structural transformation. How can “3 groups of 4” be the same as “4 groups of 3”? As embodied practices, they are entirely different actions. One involves three iterations of counting to four; the other involves four iterations of counting to three.

3.3.1 A Formal Glimpse: The Paradox of Internalized Commutation

The challenge of understanding such properties reveals the limits of any formal system that attempts to fully account for its own operations. When a system becomes complex enough to talk about its own rules, it can generate paradoxes. By “arithmetizing” the processes of addition and multiplication (encoding them as numbers, akin to Gödel’s method) and applying a fixed-point diagonalization argument, it is possible to derive a formal sentence that, in effect, asserts its own failure to commute under certain interpretations. One can derive an equation symbolically represented as:

$$\gamma \leftrightarrow \neg \text{Comm}(\text{name}(\gamma)) \quad (3.1)$$

In plain language, this formula can be interpreted as saying: “This statement is true if and only if the commuted version of the operation involving this statement is not true.” The system generates a statement that effectively declares, “*My own commutative transformation is not equivalent to me.*”

This paradoxical outcome suggests that a completely internalized and self-referential commutative operation cannot always be defined without leading to such limiting statements. This brings us into the realm of modal logic. The laws of arithmetic, like commutativity, are not contingent facts; they are *necessary* commitments for the system to be coherent. The attempt to ground this necessity from within the system itself reveals the system's own inherent incompleteness. It shows that the normative force of such laws cannot be fully captured by a purely formal, object-based ontology. The “truth” of commutativity is not a feature of the world, but a necessary condition for the coherence of our mathematical language game.

3.3.2 The Emergent Automaton: Objective Validity

The result of this complex process – of starting with a subjective, embodied practice and structuring it through normative rules of inference that exclude error – is a stable, repeatable, and objective procedure. This procedure, which embodies the norms of the practice, can be modeled as a *Finite State Automaton (FSA)*, like those in the Hermeneutic Calculator app ([savich_hermeneutic_nodate](#)). An FSA for the C2C strategy for multiplication would require:

- **States:** A start state (q_0), a state for counting items within the current group ($q_{\text{count_items}}$), a state for transitioning to the next group ($q_{\text{next_group}}$), and an accept state (q_{accept}) once the total count is achieved.
- **Variables/Counters:** To keep track of the number of groups counted (GroupCounter), items counted within the current group (ItemCounter), the accumulated total (TotalCounter), the size of each group (GroupSize), and the total number of groups (TotalGroups).

The automaton would transition between these states based on the input (the factors) and the status of its counters (e.g., moving to $q_{\text{next_group}}$ once ItemCounter equals GroupSize). This automaton is the *objective* pole of our triad. It is a third-person, formal description of the practice. Its validity is alethic in the sense that it makes a claim about what is the case: this specific, rule-governed procedure, when followed, correctly yields a result that is consistent with the normative framework. It is a formal object that emerges from, and gives structure to, the underlying subjective and normative dimensions. It is the form of the practice made explicit.

3.4 Conclusion: Operations as a Triadic Unity

Throughout this chapter, the argument has been that mathematical operations are not mere procedures applied to pre-existing objects but are normatively regulated practices emerging from embodied engagement with the world.

- The *subjective* experience of doing (e.g., the felt rhythm of C2C) provides the initial, meaningful content of an operation. It is a first-person grounding of mathematical thought.
- The *normative* framework of incompatibility semantics provides the rules of the game. It is a second-person structure of commitments and entitlements, defining coherence by systematically excluding error.
- The *objective* structure of the automaton provides the formal, repeatable, and communicable form of the practice. It is a third-person, explicit representation of a normatively governed activity.

The critical arithmetic I propose is not a closed formal system but an open-ended, pre-formal practice that is continuously evolving through dialogue and critique. It is a practice that values the agency of learners and the creative potential of mathematical thought. It resists the scientific impulse to reduce the rich, messy, human process of mathematical meaning-making to a single, “correct,” disembodied algorithm. Embracing this vision – seeing operations as a synthesis of subjective feelings, normative commitments, and objective structures – moves mathematics education toward a more emancipatory stance, fostering not just technical proficiency, but also rigorous thinking, creative exploration, and humane flourishing.

3.5 Appendix: The Hermeneutic Calculator

I am building a unified, testable theory of how students *develop* arithmetic understanding – not just a catalog of strategy names, but a developmental map of how those strategies *emerge, elaborate, invert, and nest*. My target is to formalize roughly 25 student-invented strategies across addition, subtraction, multiplication, and division, showing how each one is algorithmically constructed from prior embodied practices.

3.5.1 Choreography as Computation

I treat each strategy as **written choreography for embodied cognition**. A formal automaton (register machine, bounded DPDA, or related model) becomes a script for the temporal unfolding of thought: initialize, transform, check, recurse, terminate. The power of this framing is that it preserves *how* a student actually moves through a calculation – counting up, pausing at a boundary, decomposing a number – rather than replacing those moves with opaque symbolic shortcuts.

3.5.2 Two Fundamental Movements

I analyze student action through a dialectic of temporal structure: 1. **Temporal Compression (Sublation / Recollection)**: Unitizing many micro-acts into a larger cognitive unit (ten ones $\rightarrow$ one ten; 3 base jumps $\rightarrow$ a single composite stride). Compression accelerates flow. 2. **Temporal Decompression (Determinate Negation)**: Strategically undoing or expanding a composite to restore fine control (borrowing a ten; splitting 5 into $2 + 3$ in RMB; decomposing a factor for distributive reasoning). Fluency grows as students coordinate these movements, learning *when* to expand and *when* to re-compress.

3.5.3 Fractal Architecture: Iterative Core + Strategic Shell

Across strategies I repeatedly recover the same **fractal pattern**: * **Iterative Core**: A minimal loop (initialize $\rightarrow$ step $(+1, -1, +Base, +Chunk) \rightarrow$ condition check). Counting by ones, skip counting, and accumulation loops in division all instantiate this engine. * **Strategic Shell**: A supervisory layer that *prepares*, *optimizes*, or *transforms* the problem so that the core runs fewer or cognitively lighter iterations. RMB, Rounding & Adjusting, Chunking, Sliding, Distributive and Inverse Distributive Reasoning all wrap the core with analysis (e.g., “find gap K ”, “split factor”, “slide both numbers”). Because the shell often *invokes* the core as a subroutine (e.g., CountUpToBase, CountBackK), the global structure becomes self-similar: strategies *contain* (and sometimes nest) earlier strategies. This produces a genuine computational fractal – not metaphorical flourish, but recurrence of the same control schema at different conceptual scales.

To illustrate this, I provide a transcript from (Carpenter1999) that demonstrates a strategy that Amy Hackenberg calls *rearranging to make bases* (HackenbergCourseNotes).

- **Teacher:** Lucy is eight fish. She buys five more fish. How many fish will Lucy have then?

- **Sarah:** 13.
- **Teacher:** How'd you get 13?
- **Sarah:** Well, because eight plus two is ten, but then two plus three is five. And she wants to buy five more fish. So you take care of two, and you need to add three more. And so I add three more, and you get 13.

Notation Representing Sarah's Solution:

$$\begin{aligned}
 8 + 5 &= \square \\
 8 + 2 &= 10 \\
 2 + 3 &= 5 \\
 8 + 5 &= 10 + 3 \\
 8 + 5 &= 13
 \end{aligned}$$

Description of Strategy:

Objective: Rearranging to Make Bases (RMB) means shifting the extra ones from one addend over to the other so that one of the numbers becomes a complete multiple of the base (a whole “group” of that base). This rearrangement simplifies the addition process because there are established patterns for adding an exact multiple of the base. In other words, when you add a full group of base units to a number, the ones digit stays the same while only the digit representing the base (like the tens place) increases.

3.5.4 The Formal Model

To model this strategy as an *elaboration of counting*, I deploy an automaton called a Register Machine. Crucially, it determines the gap (K) and the remainder (R) using iterative counting, reflecting how a student might derive these values without relying on abstract subtraction. The transition table and tuple for this machine are written below.

The Fractal Choreography of Arithmetic

(Exemplar: Rearranging to Make Bases)

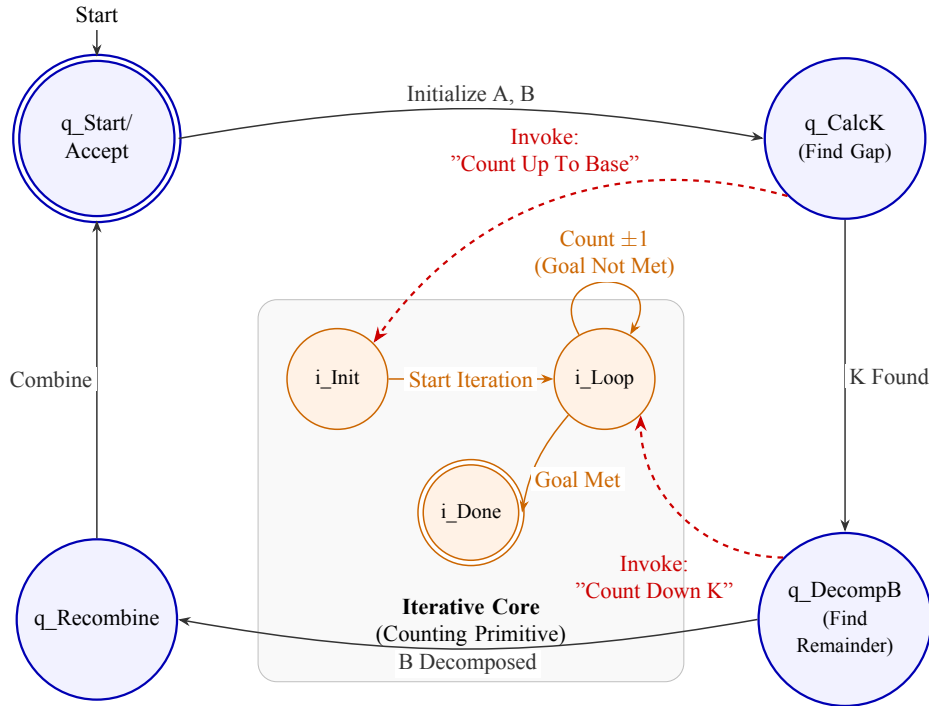


Figure 3.1: Visualization of the nested automata, illustrating Algorithmic Elaboration. The strategic states of the outer automaton (Strategic Shell) are realized by invoking the inner automaton (Iterative Core).

3.5.5 Mechanisms of Elaboration

I observe three progressive forms of algorithmic elaboration: 1. **Compression of Action:** Replacing many $+1$ steps with $+Base$ or $+StructuredChunk$ (COBO, Chunking). 2. **Optimization of Iteration:** Dynamically computing the *size* of a future stride (RMB gap K ; Chunking's bridging chunk; Sliding's constant difference) before acting – analyze then accelerate. 3. **Structural Transformation:** Rewriting the problem space (Rounding detour + compensation; Distributive split; Sliding invariance; Inverse Distributive decomposition of dividend). Advanced strategies chain these moves (e.g., Rounding = transformation $\rightarrow$ compressed addition $\rightarrow$ compensatory inverse steps).

3.5.6 Inversion of Practice

Subtraction fluency emerges not by inventing alien procedures but by **inverting or repurposing** addition shells: Missing Addend reframes subtraction as forward accumulation; Counting Back mirrors Counting On; Sliding preserves difference across a translation; Borrowing reverses carry (decompression of a prior subtraction). Division analogues (Dealing by Ones vs. Coordinating Two Counts; Inverse Distributive Reasoning) continue the same inversion logic.

3.5.7 Collaboration and Iterative Refinement

This project is explicitly *collaborative* with an AI assistant. I bring student transcripts, pedagogical insight, and theoretical intent; the assistant supplies relentless formal scrutiny – flagging mis-specified state sets, hidden non-determinism, premature algebraic assumptions, or missing termination guarantees. A typical refinement cycle: 1. Draft informal description from transcript. 2. Specify automaton (states, registers, transitions) in a first-pass formalism. 3. Implement executable prototype (Python) to test determinism, termination, and behavioral alignment with the transcript (sequence reconstruction like “46, 56, 66, ...”). 4. Trace failures (e.g., an early Rounding model produced an infinite loop after overshoot; an initial Chunking diagram hid the cognitive search for K) and revise. 5. Re-abstract the corrected machine into concise LaTeX-friendly specification. This loop ensures every claimed cognitive choreography *runs* – a falsifiability and reproducibility standard often missing in purely diagrammatic accounts.

3.5.8 Why Executable Formal Models Matter

An executable automaton does four things for me: * **Validity Check:** Catches hidden cycles or unreachable states. * **Phenomenological Fidelity:** Lets me align generated action traces with verbatim student utterances. * **Comparative Anatomy:** Normalizes different strategies into a shared tuple structure so I can map elaboration edges precisely. * **Pedagogical Insight:** Identifies which internal subroutines (e.g., “CountBackK”) must be instructionally stabilized before a composite strategy will consolidate.

3.5.9 Algorithmic Elaboration (Brandom Frame)

Following Robert Brandom, I treat these developments as **algorithmic elaborations**: later practices are *PP-sufficient* expansions of earlier ones – achieved by reorganizing, nesting, or inverting existing abilities rather than importing foreign primitives. Some strategies become **LX** relative to prior practice: they both derive from and make explicit what was implicit (RMB makes base boundaries explicit; Borrowing renders the reversibility of carry explicit; Distributive Reasoning makes latent additivity across factors explicit).

3.5.10 Scope of This Document

Below I give each strategy a uniform template: description, formal specification, choreography (compression/decompression dynamics), and genealogical lineage. I remove historical critique and raw code to foreground the structural logic while preserving testability through the already verified prototypes. The introduction you are reading consolidates the nuance of origin (embodiment), evolution (fractal elaboration), collaboration (human + AI), and rigor (execution + revision) from the longer source manuscript without duplicating passages.

3.6 Hermeneutic Calculator: Strategy Formalizations

This draft reorganizes the strategies as primary sections. Each section supplies:

1. Phenomenological description (student-facing practice).
2. Formal automaton / register-machine specification in LaTeX-friendly notation.
3. Core choreography (temporal compression/decompression dynamics).
4. Algorithmic elaboration lineage (what primitives it builds upon).

All implementation details (Python prototypes) and historical critiques have been removed. Mathematical symbols are formatted for Pandoc → LaTeX conversion.

Notation (uniform across strategies):

- $M = (Q, V, \delta, q_0, F)$: machine with states Q , registers (or variables) V , transition function δ , start state q_0 , accepting states F .
- When convenient, we use auxiliary internal variables; these are included in V implicitly.
- Counting primitives: Count Up (+1), Count Back (−1) regarded as atomic embodied actions.
- Temporal Compression: synthesizing many unit actions into a higher-order unit (e.g., a “ten”).
- Temporal Decompression: strategic expansion of a unit into constituent parts.

3.7 Counting and Counting On

Description. Sequential unit counting within a bounded base-10 place-value structure (0 – 999). Embodied iterations (“ticks”) increment units, propagate carries (sublation) into tens and hundreds.

Formal Model (Sketch). Deterministic PDA (bounded) or 3-register counter. For LaTeX exposition we specify a DPDA tuple:

$$M_{count} = (Q, \Sigma, \Gamma, \delta, q_{start}, Z_0, F)$$

with place-value stack symbols U_i, T_j, H_k . The transition function δ is defined as follows:

Current State	Input	Top of Stack	Next State	Action (Stack)	Interpretation
q_{start}	ε	Z_0	q_{idle}	Push(U_0, T_0, H_0)	Initialize count to 0.
q_{idle}	tick	U_n ($n < 9$)	q_{idle}	Pop; Push(U_{n+1})	Increment units.

Current State	Input	Top of Stack	Next State	Action (Stack)	Interpretation
q_{idle}	tick	U_9	q_{inc_tens}	Pop	Unit overflow, carry to tens.
q_{inc_tens}	ε	T_m ($m < 9$)	q_{idle}	Pop; Push(T_{m+1}, U_{10})	Increment tens, reset units.
q_{inc_tens}	ε	T_9	$q_{inc_hundreds}$	Pop	Ten overflow, carry to hundreds.
$q_{inc_hundreds}$	ε	H_k ($k < 9$)	q_{idle}	Pop; Push(H_{k+1}, T_{10})	Increment hundreds, reset lower places.
$q_{inc_hundreds}$	ε	H_9	q_{halt}	Pop; Push(H_0, T_0, U_0)	Counter overflow.

Choreography. Carry = temporal compression: ten unit steps recollected as one higher unit. Borrow (in inverse counting) is temporal decompression.

Elaboration Lineage. Primitive for all subsequent additive, subtractive, multiplicative, and divisional strategies.

3.8 Rearranging to Make Bases (RMB)

Description. For $A + B$, identify gap K from A to next base (e.g., 10, 100), decompose $B = K + R$, form $A' = A + K$ (a base), then compute $A' + R$.

Machine.

$$M_{RMB} = (Q, V, \delta, q_0, F)$$

with

$$Q = \{q_{start}, q_{calcK}, q_{decompose}, q_{recombine}, q_{accept}\}$$

$$V = \{A, B, K, A', R\}$$

Transition Function (δ):

Current State	Condition	Next State	Action	Interpretation
q_{start}	-	q_{calcK}	$K \leftarrow 0;$ $A_{temp} \leftarrow A$	Initialize.
q_{calcK}	$A_{temp} <$ $NextBase(A)$	q_{calcK}	$A_{temp} \leftarrow$ $A_{temp} + 1;$ $K \leftarrow K + 1$	Count up to find gap K .
q_{calcK}	$A_{temp} ==$ $NextBase(A)$	$q_{decompose}$	$A' \leftarrow A_{temp}$	Gap found. Store new base A' .
$q_{decompose}$	$K > 0$	$q_{decompose}$	$B \leftarrow B - 1;$ $K \leftarrow K - 1$	Decompose B by transferring K .
$q_{decompose}$	$K == 0$	$q_{recombine}$	$R \leftarrow B$	Remainder R is what's left of B .
$q_{recombine}$	-	q_{accept}	Output $A' + R$	Combine new base and remainder.

Choreography. Decompression (splitting B) enables immediate compression (forming base A').

Lineage. Elaborates Counting Up + Counting Down primitives; anticipates strategic boundary manipulation used later in Rounding, Chunking, Sliding.

3.9 COBO (Counting On by Bases then Ones)

Description. For $A+B$, decompose $B = b \cdot Base + r$; iterate base jumps ($+Base$) then unit steps ($+1$).

Machine. M_{COBO} with states $\{q_{start}, q_{bases}, q_{ones}, q_{accept}\}$ and registers $\{Sum, BaseCounter, On\}$

Transition Function (δ):

Current State	Condition	Next State	Action	Interpretation
q_{start}	-	$q_{initialize}$	Read A, B	Start.
$q_{initialize}$	-	q_{add_bases}	$Sum \leftarrow A;$ $BaseCounter \leftarrow Sum.$ $B // Base;$ $OneCounter \leftarrow B.$ B $(\text{mod } Base)$	Initialize Decompose
q_{add_bases}	$BaseCounter > q_{add_bases}$ 0	q_{add_bases}	$Sum \leftarrow$ $Sum + Base;$ $BaseCounter \leftarrow$ $BaseCounter -$ 1	Add one Base unit (Loop).
q_{add_bases}	$BaseCounter = q_{add_ones}$ 0	q_{add_ones}	-	All bases added. Transition.
q_{add_ones}	$OneCounter > q_{add_ones}$ 0	q_{add_ones}	$Sum \leftarrow$ $Sum + 1;$ $OneCounter \leftarrow$ $OneCounter -$ 1	Add one unit (Loop).
q_{add_ones}	$OneCounter = q_{accept}$ 0	q_{accept}	Output Sum	All ones added. Accept.

Choreography. Two-phase rhythm: compressed temporal blocks (bases) followed by decompressed fine resolution (ones).

Lineage. Builds on counting; prepares for Chunking and Rounding by habituating base jumps.

3.10 Rounding and Adjusting (Addition)

Description. Select addend closer to next base: round up $A \rightarrow A' = A + K$, compute $A' + B$, then adjust back: $(A' + B) - K$.

Machine. States $\{q_{start}, q_{calcK}, q_{add}, q_{adjust}, q_{accept}\}$; registers $\{A, B, K, A', Temp, Result\}$.
Transition Function (δ):

Current State	Subroutine / Action	Next State	Interpretation
q_{start}	Read A, B ; Heuristic select $Target$	q_{calcK}	Start. Select number closer to the next base.
q_{calcK}	Count Up To Base($Target$) $\rightarrow K, A_{rounded}$	q_{add}	Determine K by counting up from $Target$.
q_{add}	COBO($A_{rounded},$ Other) $\rightarrow TempSum$	q_{adjust}	Add Other to the rounded A .
q_{adjust}	Count Back($TempSum, K$) $\rightarrow Result$	q_{accept}	Adjust by counting back K .

Choreography. Strategic temporal detour: initial decompression (deriving K) enables major compression (base addition), followed by inverse correction.

Lineage. Elaborates RMB (boundary anticipation) and COBO (base efficiency); introduces explicit compensation schema.

3.11 Chunking (Addition)

Description. Decompose B into large base chunk + strategic residual chunks to force successive bases: $B = B_{base} + K + R$ where K bridges current sum to next base.

Transition Function (δ):

Current State	Condition	Next State	Action	Interpretation
q_{init}	-	$q_{addBase}$	$Sum \leftarrow A;$ Decompose B into B_{base}, B_{ones}	Initialize Sum. Decompose B .
$q_{addBase}$	-	q_{calcK}	$Sum \leftarrow$ $Sum + B_{base}$	Add the entire base chunk at once.
q_{calcK}	$Sum <$ $NextBase(Sum)$	q_{calcK}	$Sum \leftarrow$ $Sum + 1;$ $K \leftarrow K + 1$	Iteratively find gap K to next base.
q_{calcK}	$Sum ==$ $NextBase(Sum)$	q_{applyK}	-	Gap found.
q_{applyK}	$B_{ones} \geq K$	q_{calcK}	$Sum \leftarrow$ $Sum + K;$ $B_{ones} \leftarrow$ $B_{ones} - K$	Add strategic chunk K . Loop back.
q_{applyK}	$B_{ones} < K$	$q_{finishR}$	-	Not enough ones for full chunk.
$q_{finishR}$	-	q_{accept}	$Sum \leftarrow$ $Sum + B_{ones}$	Add remaining residue.

Choreography. Iterative cycle: (1) large compression via aggregated base, (2) micro decompression to find K , (3) re-compression to new base, (4) terminal residue.

Lineage. Synthesizes COBO (bulk bases) + RMB (strategic gap finding).

3.12 Subtraction Chunking (Three Orientations)

Given $M - S = D$.

A. Backwards by Part (Take-Away). Sequentially subtract decomposed parts of S (place value or strategic chunks) from M .

B. Forwards from Part (Missing Addend). Treat as $S + D = M$; Count Up (RMB logic) accumulating D .

C. Backwards to Part (Distance Down To). Count Back from M toward S using strategic base landings; accumulate distance.

Each orientation is a register machine. Below are the key transition schemas.

A. Backwards by Part (Take-Away): $V = \{CurrentValue, S_{rem}\}$

State	Condition	Action
q_{init}	-	$CurrentValue \leftarrow M;$ $S_{rem} \leftarrow S$
q_{chunk}	$S_{rem} > 0$	$Chunk \leftarrow$ $Decompose(S_{rem});$ $CurrentValue \leftarrow$ $CurrentValue -$ $Chunk;$ $S_{rem} \leftarrow S_{rem} - Chunk$
q_{chunk}	$S_{rem} == 0$	$Accept CurrentValue$

B. Forwards from Part (Missing Addend): $V = \{CurrentValue, Distance\}$

State	Condition	Action
q_{init}	-	$CurrentValue \leftarrow S;$ $Distance \leftarrow 0$
q_{chunk}	$CurrentValue < M$	$Chunk \leftarrow$ $CalcStrategicChunk(CurrentValue, M);$ $CurrentValue \leftarrow$ $CurrentValue +$ $Chunk; Distance \leftarrow$ $Distance + Chunk$
q_{chunk}	$CurrentValue == M$	$Accept Distance$

C. Backwards to Part (Distance Down To): $V = \{CurrentValue, Distance\}$

State	Condition	Action
q_{init}	-	$CurrentValue \leftarrow M;$ $Distance \leftarrow 0$
q_{chunk}	$CurrentValue > S$	$Chunk \leftarrow$ $\text{CalcStrategicChunk}(CurrentValue, S);$ $CurrentValue \leftarrow$ $CurrentValue -$ $Chunk; Distance \leftarrow$ $Distance + Chunk$
q_{chunk}	$CurrentValue == S$	Accept $Distance$

Choreography. Orientation selects temporal direction; strategies B and C exploit boundary compression via RMB subroutines.

3.13 Subtraction COBO / CBBO

COBO (Missing Addend). Start at S , perform base jumps toward M (without overshoot), then ones; distance accumulated is D .

CBBO (Counting Back). Start at M , subtract base units (from decomposed S) then ones; final position is D .

Machines. Two dual register machines are defined.

COBO (Missing Addend): $V = \{CurrentValue, Distance, Target\}$

State	Condition	Action
q_{init}	-	$CurrentValue \leftarrow S;$ $Distance \leftarrow 0;$ $Target \leftarrow M$
q_{add_bases}	$CurrentValue +$ $Base \leq Target$	$CurrentValue \leftarrow$ $CurrentValue + Base;$ $Distance \leftarrow$ $Distance + Base$
q_{add_bases}	$CurrentValue +$ $Base > Target$	transition to q_{add_ones}

State	Condition	Action
q_{add_ones}	$CurrentValue < Target$	$CurrentValue \leftarrow$ $CurrentValue + 1;$ $Distance \leftarrow$ $Distance + 1$
q_{add_ones}	$CurrentValue == Target$	Accept $Distance$

CBBO (Counting Back): $V = \{CurrentValue, BaseCounter, OneCounter\}$

State	Condition	Action
q_{init}	-	$CurrentValue \leftarrow M;$ Decompose S into $BaseCounter, OneCounter$
q_{sub_bases}	$BaseCounter > 0$	$CurrentValue \leftarrow$ $CurrentValue - Base;$ $BaseCounter \leftarrow$ $BaseCounter - 1$
q_{sub_bases} q_{sub_ones}	$BaseCounter == 0$ $OneCounter > 0$	transition to q_{sub_ones} $CurrentValue \leftarrow$ $CurrentValue - 1;$ $OneCounter \leftarrow$ $OneCounter - 1$
q_{sub_ones}	$OneCounter == 0$	Accept $CurrentValue$

Choreography. Directional inversion of the same two-phase rhythm (bases $\rightarrow$ ones). Overshoot detection acts as control boundary in COBO.

3.14 Subtraction Decomposition (Borrowing)

Description. Left-to-right: subtract higher place (tens), detect insufficiency in lower place, decompose (borrow) one higher unit into base smaller units, then subtract ones.

Transition Function (δ):

Current State	Condition	Next State	Action	Interpretation
q_{init}	-	q_{sub_bases}	Decompose M, S into place values R_T, R_O, S_T, S_O .	Initialize registers.
q_{sub_bases}	-	q_{check_ones}	$R_T \leftarrow R_T - S_T$	Subtract the bases (Tens).
q_{check_ones}	$R_O \geq S_O$	q_{sub_ones}	-	Sufficient ones. No borrow needed.
q_{check_ones}	$R_O < S_O$	$q_{decompose}$	-	Insufficient ones. Borrow.
$q_{decompose}$	$R_T > 0$	q_{sub_ones}	$R_T \leftarrow R_T - 1;$ $R_O \leftarrow R_O + Base$	Decompose (borrow) one ten.
q_{sub_ones}	-	q_{accept}	$R_O \leftarrow R_O - S_O;$ Result $\leftarrow R_T \cdot Base + R_O$	Subtract ones and combine result.

Choreography. Inversion of sublation: temporal decompression of a ten into ten ones to restore operability.

Lineage. Builds on internalized carry (from counting) now executed in reverse.

3.15 Subtraction Rounding and Adjusting

Description. Dual rounding (e.g., $M \rightarrow M'$ down, $S \rightarrow S'$ down) yields simplified $M' - S'$, then contrasting compensations: add K_M , subtract K_S .

Transition Function (δ):

Current State	Action	Next State	Interpretation
q_{start}	Read M, S	q_{roundM}	Start.
q_{roundM}	$M' \leftarrow$ RoundDown(M); $K_M \leftarrow M - M'$	q_{roundS}	Round M down. Store adjustment K_M .
q_{roundS}	$S' \leftarrow$ RoundDown(S); $K_S \leftarrow S - S'$	$q_{subtract}$	Round S down. Store adjustment K_S .
$q_{subtract}$	$Temp \leftarrow M' - S'$	$q_{adjustM}$	Calculate intermediate result.
$q_{adjustM}$	$Temp \leftarrow$ $Temp + K_M$	$q_{adjustS}$	Compensate for M (Add back).
$q_{adjustS}$	$Result \leftarrow$ $Temp - K_S$ (via chunking)	q_{accept}	Compensate for S (Subtract).

Choreography. Opposed adjustments highlight subtraction asymmetry: modification of minuend vs. subtrahend impacts result in inverse directions.

Lineage. Integrates rounding (addition strategy) and inverse compensation sequencing.

3.16 Subtraction Sliding (Constant Difference)

Description. Find K so that $S + K$ is a base (or friendly) number; compute $(M + K) - (S + K)$ exploiting invariance: $M - S = (M + K) - (S + K)$.

Transition Function (δ):

Current State	Action	Next State	Interpretation
q_{start}	Read M, S	q_{calcK}	Start. Target S for adjustment.
q_{calcK}	$K \leftarrow$ CountUpToBase(S)	q_{slide}	Iteratively find the gap K .

Current State	Action	Next State	Interpretation
q_{slide}	$M' \leftarrow M + K;$ $S' \leftarrow S + K$	$q_{subtract}$	Apply the slide K to both M and S .
$q_{subtract}$	$Result \leftarrow$ $M' - S'$	q_{accept}	Perform the simplified subtraction.

Choreography. Up-front decompression (deriving K) enables single compressed subtraction against a base-aligned subtrahend.

Lineage. Extends RMB gap-finding; anticipates relational “distance” framing central to subtraction fluency.

3.17 Commutative Reasoning (Multiplication Optimization)

Description. For $A \times B$, evaluate heuristic difficulty of (A, B) vs (B, A) ; select orientation minimizing cognitive load (iteration count & skip difficulty), then perform iterative addition (skip counting).

Transition Function (δ):

Current State	Condition / Heuristic	Next State	Action
$q_{evaluate}$	$H(B, A) <$ $H(A, B)$	$q_{repackage}$	-
$q_{evaluate}$	(Otherwise)	q_{calc}	$Groups \leftarrow A;$ $Items \leftarrow B$
$q_{repackage}$	-	q_{calc}	$Groups \leftarrow B;$ $Items \leftarrow A$
q_{calc}	-	q_{accept}	$Total \leftarrow$ $IterativeAdd(Groups, Items)$

Choreography. Meta-level selection precedes execution; commutative symmetry exploited for temporal compression.

Lineage. Builds on C2C / Skip Counting; introduces optimization layer.

3.18 Coordinating Two Counts (C2C)

Description. Foundational multiplication: nested counting – items within group, groups within total; total $T = N \cdot S$ emerges from exhaustive unit enumeration.

Transition Function (δ):

Current State	Condition	Next State	Action
q_{init}	-	q_{checkG}	$G \leftarrow 0, I \leftarrow 0, T \leftarrow 0$
q_{checkG}	$G < N$	$q_{countItems}$	-
q_{checkG}	$G == N$	q_{accept}	Output T
$q_{countItems}$	$I < S$	$q_{countItems}$	$I \leftarrow I + 1, T \leftarrow T + 1$
$q_{countItems}$	$I == S$	$q_{nextGroup}$	-
$q_{nextGroup}$	-	q_{checkG}	$G \leftarrow G + 1, I \leftarrow 0$

Choreography. Maximal temporal decompression (no compression yet); establishes structural scaffold for later compression (skip counting, distributive reasoning).

Lineage. Direct elaboration of counting primitives into nested loops.

3.19 Conversion to Bases and Ones (CBO Multiplication)

Description. Redistribute units among groups so that many groups become exact base multiples, leaving a compact residual: $(k \cdot Base) + r$.

Transition Function (δ):

Current State	Condition	Next State	Action
q_{init}	-	q_{select_source}	Initialize <i>Groups</i> array with value S .
q_{select_source}	$N > 0$	$q_{transfer}$	Select a <i>SourceIdx</i> .
$q_{transfer}$	$Groups[Source] > 0$ AND not all targets full	$q_{transfer}$	Transfer 1 unit from <i>Source</i> to next available <i>Target</i> .
$q_{transfer}$	(Source empty OR all targets full)	$q_{finalize}$	-
$q_{finalize}$	-	q_{accept}	Total $\leftarrow \sum Groups$.

Choreography. Proactive sublation: simultaneous decompression (source group) and compression (targets) to manufacture base units early.

Lineage. Multiplicative analogue of RMB and addition Chunking with explicit inter-group transfers.

3.20 Distributive Reasoning (Multiplication)

Description. Decompose $S = S_1 + S_2$ (heuristically “easy” numbers), compute NS_1 and NS_2 (skip counting or compressed methods), then sum.

Transition Function (δ):

Current State	Action	Next State
q_{split}	$S_1, S_2 \leftarrow \text{HeuristicSplit}(S)$	q_{P1}
q_{P1}	$P_1 \leftarrow \text{IterativeAdd}(N, S_1)$	q_{P2}
q_{P2}	$P_2 \leftarrow \text{IterativeAdd}(N, S_2)$	q_{sum}
q_{sum}	$Total \leftarrow P_1 + P_2$	q_{accept}

Choreography. Temporal decompression (factor split) followed by parallelizable compressed sub-calculations and final recombination.

Lineage. Extends skip counting with heuristic structural decomposition; precursor to algebraic distributivity recognition.

3.21 Dealing by Ones (Division – Sharing)

Description. Partitive division: distribute single units round-robin into N groups until total T exhausted; per-group size S emerges.

Transition Function (δ):

Current State	Condition	Next State	Action
q_{init}	-	q_{deal}	$Remaining \leftarrow T$; Initialize $Groups$ array to 0s.
q_{deal}	$Remaining > 0$	q_{deal}	$Groups[idx] \leftarrow Groups[idx] + 1$; $Remaining \leftarrow Remaining - 1$; $idx \leftarrow (idx + 1) \pmod{N}$
q_{deal}	$Remaining == 0$	q_{accept}	Output $Groups[0]$

Choreography. Maximal temporal decompression; rhythmic rounds establish invariant increase pattern (foundation for later compression insights).

Lineage. Inversion of C2C perspective (constructing equal groups from total rather than composing total from groups).

3.22 Inverse Distributive Reasoning (Division)

Description. Measurement division T/S : decompose T into known multiples of S : $T = \sum_i (m_i S)$; quotient = $\sum_i m_i$.

Transition Function (δ):

Current State	Condition	Next State	Action
q_{init}	-	q_{search}	$Remaining \leftarrow T$; $TotalQ \leftarrow 0$; Load KB for S .
q_{search}	Found (P_T, P_Q) in KB where $P_T \leq Remaining$	q_{apply}	Select largest such (P_T, P_Q) .
q_{search}	No suitable fact found	q_{accept}	Output $TotalQ$.
q_{apply}	-	q_{search}	$Remaining \leftarrow Remaining - P_T$; $TotalQ \leftarrow TotalQ + P_Q$.

Choreography. Temporal compression via retrieval of pre-compressed multiplication facts; loop greedily subtracts largest available chunk.

Lineage. Inversion of Distributive Reasoning in multiplication (switch from constructing product to decomposing dividend).

3.23 Using Commutative Reasoning (Division via Iterated Accumulation)

Description. For E/G (sharing reframed as measurement): iteratively accumulate G until total E reached; iteration count is quotient.

Transition Function (δ):

Current State	Condition	Next State	Action
q_{init}	-	$q_{iterate}$	$Acc \leftarrow 0$; $Q \leftarrow 0$.
$q_{iterate}$	-	q_{check}	$Acc \leftarrow Acc + G$; $Q \leftarrow Q + 1$.

Current State	Condition	Next State	Action
q_{check}	$Acc < E$	$q_{iterate}$	-
q_{check}	$Acc == E$	q_{accept}	Output Q .

Choreography. Symmetric inversion of repeated addition (multiplication) focusing on completion criterion instead of fixed loop count.

Lineage. Bridges between Dealing by Ones and chunk-based division (fact retrieval).

3.24 Conversion to Groups Other than Bases (CJOB Division)

Description. Leverage base decomposition of dividend T (e.g., tens & ones) plus analysis of base/divisor relation: $Base = q_1S + r_1$; process all base units in bulk, aggregate remainders, finalize.

Transition Function (δ):

Current State	Action	Next State
q_{init}	Decompose T into T_B, T_O ; $Q \leftarrow 0, R \leftarrow 0$.	$q_{analyze}$
$q_{analyze}$	$S_{inB} \leftarrow B // S$; $R_{inB} \leftarrow B \pmod{S}$.	$q_{processBases}$
$q_{processBases}$	$Q \leftarrow Q + T_B \cdot S_{inB}$; $R \leftarrow R + T_B \cdot R_{inB}$.	$q_{combineR}$
$q_{combineR}$	$R \leftarrow R + T_O$.	$q_{processR}$
$q_{processR}$	$Q \leftarrow Q + R // S$; $R \leftarrow R \pmod{S}$.	q_{accept}

Choreography. Dual decompression (dividend by base, base by divisor) $\rightarrow$ large compression (bulk quotient) $\rightarrow$ residual resolution.

Lineage. Division analogue of CBO (multiplication) and Distributive Reasoning; integrates multi-level structural analysis.

3.25 Conceptual Dependency Graph (Narrative)

Counting $\rightarrow$ (RMB, COBO) $\rightarrow$ (Chunking, Rounding & Adjusting, Sliding) $\rightarrow$ (Subtraction Inversions: COBO/CBBO, Chunking orientations, Decomposition, Rounding, Sliding) $\rightarrow$ (C2C) $\rightarrow$ (Skip Counting / implicit in COBO Multiplication) $\rightarrow$ (Commutative & Distributive Reasoning, CBO Multiplication) $\rightarrow$ (Division primitives: Dealing by Ones, Iterated Accumulation) $\rightarrow$ (Inverse Distributive Reasoning, Fact-Based Decomposition) $\rightarrow$ (CFOB Division).

Each arrow denotes an algorithmic elaboration where prior compressed units or reversible decompositions become callable subroutines.

3.26 Temporal Dynamics Summary

- **Primitive Decompression:** Counting by ones; Dealing by Ones; C2C (inner loop).
- **First Compression Layer:** COBO (bases as units); subtraction COBO/CBBO; iterative accumulation for division.
- **Strategic Boundary Forcing:** RMB, Chunking, Sliding, Rounding (anticipatory manipulation of base thresholds).
- **Structural Decomposition / Synthesis:** Distributive Reasoning, Inverse Distributive Reasoning, CBO (Multiplication & Division), CFOB.

3.27 Glossary of Symbols

- *Base*: Typically 10 (extendable to other positional bases).
- *K*: Gap to next base (RMB, Rounding, Chunking, Sliding).
- *R*: Remainder after decomposition or partial processing.
- *S₁, S₂*: Split components of a factor (Distributive Reasoning).

- *Groups, Items*: Multiplicative roles after commutative optimization.
 - *Remaining*: Unprocessed portion of a dividend in division strategies.
 - *Q*: Quotient / accumulated result in division; also generic state set symbol context-dependent.
 - *Acc*: Accumulated total during iterative division.
-

3.28 Fractions

I now turn to fractions. Rather than recreating the wheel, I will demonstrate how to transpose extant research in the field of math education into a pragmatic key. The rich psychological models of a student's mathematical development, grounded in Leslie P. Steffe's radical constructivism, are translated into a formal, executable automaton. The goal is to re-key the descriptive insights of constructivism into a pragmatic framework that aligns with the spirit of Robert Brandom's analytic pragmatism, where knowing-how is made explicit in the form of a communicable knowing-that. I model the cognitive schemes of a student, "Jason," as a set of nested finite automata, moving from the psychological description of his mental operations to a computational specification. The resulting executable model demonstrates how cognitive development, particularly the "metamorphic accommodations" identified by constructivists, can be understood as the reorganization and nesting of formal procedures. The output of the model, a structured history of its operations, serves as a formal representation of the constructed meaning of a mathematical concept.

Leslie P. Steffe's work on the mathematical development of children offers a detailed account of cognitive construction, framed within the epistemology of radical constructivism (**steffe2002new**). This perspective posits that learners actively build their own mathematical realities, not by discovering a pre-existing world, but by adapting their cognitive schemes to fit the constraints of their experience (**von2005contradictions**). The teaching experiment methodology provides a longitudinal lens through which researchers can build second-order models of this construction, creating what is termed the "mathematics of students" (**steffe2014teaching**).

My objective here is not to question the descriptive validity of this constructivist model but to transpose it into a different philosophical key. I aim to translate

the psychological description of one student's learning trajectory – that of “Jason” – into a formal, executable model. This act of translation serves a purpose aligned with analytic pragmatism: to make the implicit practices and cognitive choreography of mathematical knowing explicit in a formal structure. The resulting automaton is a pragmatic interpretation of Jason's mathematical schemes, where the focus shifts from the student's private experiential world to the public, formal specification of a viable cognitive system.

3.29 The Operational Foundation: The Explicitly Nested Number Sequence (ENS)

The foundation for Jason's later fractional knowledge was his whole-number operating system, what Steffe terms the Explicitly Nested Number Sequence (ENS) (steffe2002new). The ENS is an interiorized counting scheme that allows a child to treat numbers and number sequences as objects of thought. The power of the ENS lies in a set of core mental operations that I will treat as the primitive functions of my computational model:

- **Iteration:** The ability to take a composite unit (e.g., a “six”) as a single item and repeat it to produce larger quantities (steffe2014childrens).
- **Disembedding:** The ability to mentally isolate a numerical part from a whole without destroying the conceptual integrity of the whole (steffe2002new).
- **Coordination of Units:** The capacity to flexibly view a number, such as twelve, as a single unit of twelve, a composite unit of twelve ones, and as a part of a larger sequence (steffe2004fractional).

These operations form the iterative core of the automaton, the fundamental actions from which more complex, goal-directed schemes are built.

3.30 The Emergence of Fractional Schemes

According to the “reorganization hypothesis,” fractional knowledge emerges as an accommodation of these whole-number schemes when applied to continuous quantities (steffe2002new). My formal model captures this process by defining strategic shells – automata – that organize the core operations to solve new kinds of problems.

3.30.1 The Partitive Fractional Scheme (PFS)

Jason's first major accommodation was the construction of a Partitive Fractional Scheme (PFS), his primary tool for producing a proper fraction, such as making $\frac{3}{7}$ of a continuous unit (a "stick") (**steffe2003construction**). This scheme repurposes the ENS operations: the number sequence (1 to 7) is used as a template to *partition* the stick, one part (the $\frac{1}{7}$ unit fraction) is *disembedded*, and that part is then *iterated* three times.

I model this goal-directed activity as a finite automaton, $M_{PFS} = (Q, V, \delta, q_0, F)$, which orchestrates the core operations.

- **States (Q):** $\{q_{start}, q_{partition}, q_{disembed}, q_{iterate}, q_{accept}\}$
- **Variables (V):** {Whole, N (Denominator), M (Numerator), PartitionedWhole, UnitFraction, Result}
- **Transition Function (δ):** The function choreographs the sequence of core operations, moving from partitioning the whole, to disembedding the unit part, to iterating that part to form the final fraction.

The sequence is not merely a list of steps but a structured, goal-directed procedure.

3.30.2 Metamorphic Accommodation: Recursive Partitioning

A pivotal event in Jason's development was his spontaneous invention of a method for finding a fraction of a fraction (e.g., $\frac{3}{4}$ of $\frac{1}{4}$ of a stick), an act Steffe identifies as a "metamorphic accommodation" (**steffe2003construction**). This signaled the construction of a new operation: *recursive partitioning*. Jason could take the *result* of one partitioning operation and use it as the *input* for a subsequent one.

This cognitive leap is modeled by a second automaton, the Fractional Composition Scheme (FCS). The architecture of the FCS demonstrates a fractal-like elaboration: it is a strategic shell that calls the entire PFS automaton as a subroutine. This nesting of a previously constructed scheme to solve a more complex problem is the formal analogue of the psychological process of accommodation. The critical step in the FCS automaton is the state $q_{accommodate}$, where the output of the first PFS execution (the intermediate fraction) is formally re-assigned as the input "Whole" for the second PFS execution.

3.31 The Formal Automaton Model

The psychological descriptions are translated into a formal Python implementation. The ‘ContinuousUnit’ class represents the cognitive material, tracking not only its numerical quantity but also its operational history, thereby capturing the constructed meaning of the number. The core ENS operations are static methods, and the schemes (PFS and FCS) are implemented as state machine classes.

3.31.1 Python Implementation

```

1  import fractions
2  from typing import List, Tuple
3
4  class ContinuousUnit:
5      """Represents a continuous quantity, tracking both value and history."""
6      def __init__(self, value: fractions.Fraction, history: str = "Reference Unit"):
7          self.value = value
8          self.history = history
9      def __repr__(self):
10         return f"Unit({self.value} derived from: '{self.history}')"
11
12 class ENSOperations:
13     """The iterative core operations derived from Jason's ENS."""
14     @staticmethod
15     def partition(unit: ContinuousUnit, n: int) -> List[ContinuousUnit]:
16         new_value = unit.value / n
17         new_history = f"1/{n} part of ({unit.history})"
18         return [ContinuousUnit(new_value, new_history) for _ in range(n)]
19     @staticmethod
20     def disembed(partitioned_whole: List[ContinuousUnit]) -> ContinuousUnit:
21         return partitioned_whole[0]
22     @staticmethod
23     def iterate(unit: ContinuousUnit, m: int) -> ContinuousUnit:
24         new_value = unit.value * m
25         new_history = f"{m} iterations of [{unit.history}]"
26         return ContinuousUnit(new_value, new_history)
27
28 class PartitiveFractionalScheme:
29     """[SHELL::PFS] Automaton model for constructing proper fractions."""

```

```

30     def __init__(self):
31         self.F = {'q_accept'}
32         self.V = {}
33         self.trace = []
34     def run(self, whole: ContinuousUnit, num: int, den: int) -> ContinuousUnit:
35         # Simplified for brevity: direct implementation of the state logic
36         self.trace.append("State: q_partition -> Partitioning Whole")
37         partitioned = ENSOperations.partition(whole, den)
38         self.trace.append("State: q_disembed -> Disembedding Unit Fraction")
39         unit_fraction = ENSOperations.disembed(partitioned)
40         self.trace.append(f"State: q_iterate -> Iterating Unit Fraction {num} times")
41         result = ENSOperations.iterate(unit_fraction, num)
42         self.trace.append("State: q_accept -> PFS Complete")
43         return result
44
45 class FractionalCompositionScheme:
46     """[SHELL::FCS] Models recursive partitioning by nesting the PFS."""
47     def __init__(self):
48         self.F = {'q_accept'}
49         self.PFS = PartitiveFractionalScheme()
50         self.trace = []
51     def run(self, whole: ContinuousUnit, outer: Tuple[int, int], inner: Tuple[int, int]):
52         A, B = outer
53         C, D = inner
54         self.trace.append("State: q_inner_PFS -> Calculating inner fraction")
55         intermediate_result = self.PFS.run(whole, C, D)
56         self.trace.append("State: q_accommodate -> METAMORPHIC ACCOMMODATION")
57         new_whole = intermediate_result
58         self.trace.append("State: q_outer_PFS -> Calculating outer fraction")
59         final_result = self.PFS.run(new_whole, A, B)
60         self.trace.append("State: q_accept -> FCS Complete")
61         return final_result

```

3.31.2 Execution and Analysis

Running the model replicates Jason's cognitive choreography.

Test 1: Partitive Fractional Scheme ($\frac{3}{7}$ of a Whole)

Execution Trace:

State: q_partition -> Partitioning Whole

State: q_disembed -> Disembedding Unit Fraction
 State: q_iterate -> Iterating Unit Fraction 3 times
 State: q_accept -> PFS Complete

RESULT: Unit(3/7 derived from: '3 iterations of [1/7 part of (Reference Unit)]')

Test 2: Fractional Composition Scheme ($\frac{3}{4}$ of $\frac{1}{4}$ of a Whole)

Execution Trace:

State: q_inner_PFS -> Calculating inner fraction
 State: q_accommodate -> METAMORPHIC ACCOMMODATION
 State: q_outer_PFS -> Calculating outer fraction
 State: q_accept -> FCS Complete

RESULT: Unit(3/16 derived from: '3 iterations of
 [1/4 part of (1 iterations of [1/4 part of (Reference Unit)])]')

The execution trace makes the temporal unfolding of Jason's scheme explicit. The result of Test 2 is not the bare quantity $\frac{3}{16}$, but a quantity whose meaning is constituted by its operational history. The output string is a formal record of the nested operations that produced the result, making the constructed nature of the concept transparent. This aligns with a pragmatist view where meaning is rooted in practice.

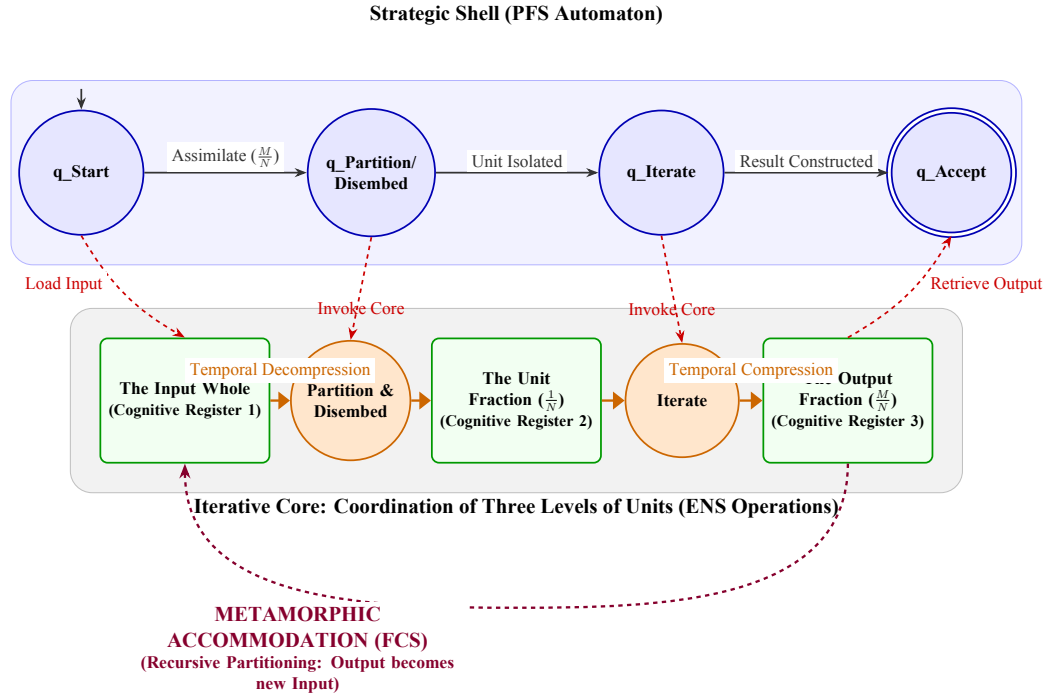


Figure 3.2: Visualization of Jason’s fractional reasoning architecture. The Strategic Shell (PFS) choreographs the Iterative Core (ENS Operations) to coordinate the Three Levels of Units (Input Whole, Unit Fraction, Output Fraction). The recursive pathway (purple) illustrates the reorganization into the Fractional Composition Scheme (FCS), where the scheme operates on its own output (fractal elaboration).

3.32 Conclusion: From Psychological Description to Pragmatic Specification

This project successfully translates the psychological description of Jason’s mathematical schemes into a formal, executable automaton. This translation re-keys radical constructivism into a pragmatic framework. While constructivism focuses on modeling the student’s viable, private reality, this formal model provides a public, verifiable specification of a cognitive practice.

The automaton does not claim to be a “true” picture of Jason’s mind. Rather, it is a formal model of the *choreography of his reasoning*. The execution trace

makes the inferential steps of his practice explicit. The “metamorphic accommodation” of recursive partitioning is modeled not as an inexplicable insight, but as a structural change in the automaton: the nesting of one goal-directed procedure within another.

In this pragmatic key, understanding a concept like “ $\frac{3}{16}$ ” is not about having a mental representation that corresponds to reality, but about possessing the structured, operational capacity to produce it. The automaton’s final output, with its embedded history, is a formal representation of this capacity. It is a piece of “knowing-that” which explicitly codifies the “knowing-how” of Jason’s mathematical practice.

Appendix: Summary of Jason’s Mathematical Schemes

Table 3.22: Synthesis of Jason’s Mathematical Schemes and Constituent Operations

Scheme / Operational State	Triggering Situation (Input)	Core Mental Operations (Transition Function)	Result (Output)
Explicitly Nested Number Sequence (ENS)	Whole-number tasks requiring part-whole reasoning.	- Iterating composite units. - Disembedding a numerical part. - Coordinating three levels of units.	A numerical quantity or relation.
Partitive Fractional Scheme	Task to construct a proper fraction ($\frac{m}{n}$) of a whole.	- Partitioning the whole into n parts. - Disembedding one unit part ($\frac{1}{n}$). - Iterating the unit part m times.	A new continuous quantity representing the fraction $\frac{m}{n}$.
Recursive Partitioning Operation	Novel task to find a fraction of a fractional part.	- Taking the result of a partitioning operation as the input for a subsequent partition.	A new unit fraction related to the whole via composition.
Fractional Composition Scheme	Generalized task to find a fraction of a fraction.	- Stabilized application of the recursive partitioning operation.	A new fractional quantity representing the product.

Chapter 4

9: Algorithmic Elaboration and History

Abstract

This chapter explores the development of mathematical concepts and procedures through the framework of algorithmic elaboration, arguing that the history of mathematics reflects a dialectical pattern in which apparent completeness generates new possibilities. Building on the previous chapter's claim that numerals function as pronouns, the discussion connects this insight to the development of critical arithmetic through Robert Brandom's concept of algorithmic elaboration. Two case studies structure the analysis: first, a reconstruction of Euclid's proof of the infinity of primes using incompatibility semantics; second, an examination of how arithmetic operations emerge from embodied metaphors, drawing on the work of George Lakoff and Rafael Núñez. These case studies reveal the limitations of simple algorithmic elaboration and introduce the concept of "pragmatic expressive bootstrapping," in which conceptual systems develop by making explicit the normative structures implicit in existing practices. The chapter challenges the traditional approach to literature review, advocating instead for understanding intellectual history as a dynamic process of conceptual development. This approach treats history not as a static repository of facts but as an ongoing conversation in which each contribution elaborates and transforms what came before.

4.1 The Misrecognition of Totality

When I proposed this book, I was interested in exploring the concept of literature reviews. In the intervening years, I wrote enough on that idea to fill a book and sated my curiosity about the topic. I found literature reviews to be terribly confusing when I was writing my dissertation. I thought it would be a gift to ink some prose devoted to understanding the process of literature review. The earth shifted under my feet, as the popularization of Large Language Models seem to make the processes of summarizing and synthesizing a more or less passive activity. However, as a practical matter, I still do not know how to do a literature review.

As I continued to write on this project, I decided that I would not write about literature reviews in the way I originally intended, but I still was not sure how to approach the topic of history and the concept of algorithmic elaboration. Instead, I will demonstrate the concept of history as an algorithmic elaboration by instantiating it within the history of mathematics. I will then discuss the limits of algorithmic elaboration, particularly in the context of dialectical moments in the history of mathematics.

Traditionally conceived, algorithms are step-by-step procedures for solving problems or performing tasks. They are foundational to computer science, mathematics, and many areas of human activity. An algorithm takes an input, processes it through a series of defined steps, and produces an output. What this definition of algorithms can miss is the underlying inferential structure of algorithms. Essentially, an algorithm is a chain of inferences. These chains of inference can be smoothed, just like someone who practices yoga can learn to flow between the poses. Of particular interest are chains of material inference involving substitution. The concept of algorithmic elaboration emerges from the practices of multiplication and subtraction.

The misrecognition at stake is the assumption that “The Literature”—the accumulated body of mathematical knowledge—is a fixed, complete totality that one must master before contributing. This reifying assumption paralyzes creative thought. It treats history as a dead object rather than a living process. Similarly, algorithms are often misrecognized as mere mechanical procedures, devoid of the rational, inferential structure that makes them intelligible and powerful. Why try to advise when I do not know? It is not that I do not care what scholars have said before me, but I could not help but feel like every time I tried to gain some purchase on the totality, I found a new gap. Some other article would fill that gap, and then I found another, and another, and another.

4.2 The First Determinate Negation: The Inferential Turn

To make this misrecognition determinate, we turn to Robert Brandom's concept of algorithmic elaboration, which reconceives algorithms not as mechanical procedures but as chains of material inference. This section introduces the tools of incompatibility semantics and shifts our understanding from static history to dynamic conceptual development.

Since I already discussed the bones of Euclid's proof in Chapter 7, I will begin my reconstruction by introducing some terms from Brandom's *incompatibility semantics* (Brandom2008). The first is the notion of an *incoherence frame*. An *incoherence frame* is a pair

$$\langle L, Inc \rangle,$$

where *Inc* specifies which sentences in the language *L* are *incoherent*. For Brandom, these are formal languages written by automata. But it does not do much harm to the notion to think of a language as a collection of sentences a qualitative researcher seeks to understand, or the way that people in general reflect on troubling interactions to try to make sense of them. Incoherence can be existentially terrifying. When I am recognized as someone who believes *x* and $\neg x$, the recognizer might understand my beliefs as evolving. I can be taken as being who is becoming. But I can also be taken as someone who cannot be trusted.

To comply with Brandom's formalization of *incompatibility*, consider an incoherent set such as *A* : "now is night" and *B* : "now is day," where both occurrences of "now" are intended to refer to the same moment. By the *persistence* axiom, simply adding more sentences to the language does not repair this incoherence.

An *incompatibility relation* *I* is defined so that two sets *X* and *Y* are *incompatible* if and only if their union is incoherent. That is, *A* is incompatible with *B* if taking them together results in an incoherence. Symbolically, $A \in I(B)$ iff $A \cup B \in Inc$.

This framework transforms our understanding of algorithmic reasoning. Algorithms are not arbitrary sequences of steps but chains of material inference where each step is entitled by preceding commitments and excludes incompatible alternatives. The incompatibility relation structures the space of possible inferences, making algorithmic elaboration a rational, norm-governed process rather than a mechanical manipulation.

4.3 Systematic Analysis: Euclid’s Proof Reconstructed

The claim “{The list of primes $\{A, B, C\}$ is complete} $\in Inc$ ” belongs to the set of incoherent claims because it generates an internal contradiction. Once we understand what prime numbers are, this claim defeats itself. The incoherence emerges through the following steps:

1. If the list $\{A, B, C\}$ is complete, then any newly constructed number must either be composite or equal to a prime already on the list.
2. The construction $N = A \times B \times C + 1$ yields a number that, whether prime or composite, forces the existence of a prime not on the list.
3. This contradicts the initial assumption of completeness, demonstrating that the claim to completeness is an element of an incoherence frame.

This reconstruction casts history into the field of iteration. By translating Euclid’s geometric language into Brandom’s vocabulary of incompatibility relations, the proof’s power comes from identifying a fundamental incoherence in the claim to completeness. The algorithmic substitution of “incoherence” for “contradiction” opens new possibilities for mathematical education. Most significantly, incompatibility semantics allows errors, mistakes, and misconceptions to be admitted into mathematical systems without destroying their coherence—these apparent problems become elements to be incorporated rather than eliminated.

Formal Reconstruction in Prolog

The conceptual reconstruction can be formalized as machine-executable rules within a Prolog-based incompatibility semantics engine. At the foundation, the system connects symbolic reasoning to computational facts through *grounded predicates*:

- `prime(N)`: Succeeds if N is a prime number.
- `composite(N)`: Succeeds if N is a composite number.
- `divides(D, N)`: Succeeds if D divides N .

Building on these, the core logic is captured in two key *material inferences*:

1. **M5 (Prime Divisor Incompatibility)**: Any prime factor G of the constructed number $N = (\prod L) + 1$ cannot be a member of the original list of primes L . This is the central incompatibility Euclid identifies.

4.4. THE DIALECTICAL TURN: LIMITS AND EMBODIED FOUNDATIONS⁹¹

2. **M4 (Completeness Definition):** If there exists a prime G that is not on the list L , then the list L is not complete.

The system models Euclid's procedure using *structural rules* that dynamically add new information:

1. **Construction:** Given the premise that list L is complete, calculate $N = (\prod L) + 1$ and trigger the next stage.
2. **Case Analysis:** Prove the goal by assuming N is prime, then again assuming N is composite. If completeness leads to incoherence in *both* cases, the assumption is proven incoherent.
3. **Prime Factorization:** In the composite case, find a prime factor G of N and add premises `prime(G)` and `divides(G, N)` to enable material inferences M4 and M5 to fire.

By combining these elements, the assertion `incoherent([n(is_complete(L))])` is proven. The engine applies construction, proceeds through case analysis, and uses forward-chaining over material inferences to arrive at the conclusion that the list is *not* complete, rendering the initial premise incoherent. This formal elaboration transforms Euclid's elegant proof into a running algorithm, demonstrating the viability of incompatibility semantics for mathematical reasoning.

4.4 The Dialectical Turn: Limits and Embodied Foundations

Algorithmic elaboration is powerful for making the implicit rules of a given practice explicit. However, conceptual history also contains dialectical moments where the system of rules itself must be transcended. These moments occur when a system of normative commitments confronts an observation that it cannot account for—an observation that is *incoherent* under its current structure.

In my dissertation, I synthesized Lakoff and Núñez's (**Lakoff2000**) embodied metaphors for mathematics with Brandom's incompatibility semantics. At first, this was a challenge, but with familiarity the process began to feel algorithmic. The seed is planted—if others want to make it grow, they can water it.

Bounded Regions and Categories Initially, proprioceptive feelings define a boundary—comprising an *interior* (inside the body) and an *exterior* (outside the body). The following rules govern the vocabulary associated with bounded regions:

1. If you are in a *bounded region*, you are not out of that *bounded region*.
2. If you are out of a *bounded region*, you are not in that *bounded region*.
3. If you are deep in a *bounded region*, you are far from being out of that *bounded region*.
4. If you are on the edge of a *bounded region*, you are close to being in that *bounded region*.

Substitution Rules for Categories To transition from bounded regions to categories, apply the following substitutions:

1. Substitute “Categories” for “Bounded regions in space.”
2. Substitute “Category members” for “Objects inside the bounded regions.”
3. Substitute “A subcategory of a larger category” for “One bounded region inside another.”

This substitution allows the body-feeling to ground the notion of categories, maintaining a coherent vocabulary transition that supports the development of mathematical concepts. By algorithmically elaborating these substitution rules, the evolution from basic proprioceptive experiences to abstract mathematical categories can be made explicit, ensuring each step maintains logical consistency and normative correctness. This approach mirrors pedagogical methods where complex mathematical vocabularies are built upon foundational concepts. Educators trace vocabulary transitions from simple to complex, allowing learners to internalize and apply new concepts seamlessly.

The limitation revealed here is that algorithmic elaboration can systematically develop concepts *within* a given framework but cannot generate the radical conceptual shifts required when the framework itself becomes inadequate. For such transformations, we need a different mechanism.

4.5 Distinguishing Two Modes of Practice Development: PEdT and AE

Before exploring pragmatic expressive bootstrapping, I need to clarify a crucial distinction that has shaped my understanding of how mathematical concepts develop. This distinction emerged from my dissertation work, where I tried to synthesize Brandom's semantic inferentialism with Lakoff and Núñez's embodied cognition research. At first, the two frameworks seemed incompatible. Brandom talks about algorithmic elaboration as if mathematical development follows a logical, predictable path. Lakoff talks about metaphorical grounding as if mathematics emerges from bodily experience. Both are right, but they describe different mechanisms.

The key understanding is that there are *two distinct ways* that practices develop from other practices. Brandom calls these *Practical Elaboration by Training* (PEdT) and *Algorithmic Elaboration* (AE) (**Brandom2008**). Grasping this distinction transformed how I think about teaching mathematics—and why I failed so spectacularly in some of my early attempts.

The Origin: Embodied Metaphors and Material Inference

Let me start with where mathematics begins: in the body. Lakoff and Núñez argue that abstract mathematical concepts are grounded in sensory-motor experience through *conceptual metaphor* (**Lakoff2000**). This is not a literary device. It is a cognitive mechanism where the inferential structure of one domain (concrete, bodily) gets mapped onto another domain (abstract, mathematical).

They identify what they call the *Four Grounding Metaphors* for arithmetic—the “4Gs.” I will explain these because they are foundational to understanding how mathematical meaning emerges from embodied practice:

Arithmetic as Object Collection This is perhaps the most intuitive metaphor. When you put two piles of objects together, you have combined them. When you remove some objects from a pile, you have subtracted. The physical act of collecting objects carries with it material inferences: if I have three apples in one hand and two apples in the other, bringing my hands together gives me five apples. This inference is good not because of logical form but because of the material reality of object collection.

The mappings are straightforward: collections of objects $\rightarrow$ numbers; putting collections together $\rightarrow$ addition; taking a collection away $\rightarrow$ subtraction. Crucially, this metaphor carries commutativity and associativity. When I pool two collections, the order does not matter. Three apples plus two apples equals two apples plus three apples because the physical act of pooling is indifferent to sequence.

Arithmetic as Object Construction This metaphor shifts the focus from collections to wholes and parts. A number is a whole made up of parts, and arithmetic operations describe how these parts fit together. This is the metaphor children use when they say “Five is two and three” or “Eight is four and four.” They are constructing the whole from its parts. This metaphor grounds understanding of decomposition, which becomes essential for subtraction with regrouping (“borrowing”).

Arithmetic as Motion Along a Path This metaphor understands numbers as locations on a path and arithmetic as movement along that path. When you add, you move forward; when you subtract, you move backward. This is the metaphor that grounds “counting on”—the strategy where a child solving $5 + 3$ starts at five and counts forward: “six, seven, eight.”

Motion Along a Path carries different inferences than Object Collection. There is a directionality, a sense of order. The path has a beginning (zero) and extends infinitely forward. This metaphor makes the successor function natural: the next number is the next step on the path.

Arithmetic as Measuring Stick The fourth grounding metaphor conceptualizes numbers as measurements and arithmetic as operations on measurements. This metaphor grounds understanding of units, fractions, and eventually real numbers. When I use a ruler, I am not counting discrete objects but measuring continuous quantity.

These four metaphors are not mutually exclusive. A competent arithmetic reasoner moves fluidly between them, drawing on whichever metaphor makes the current problem tractable. But here is the crucial point: these metaphors do not automatically yield stable mathematical practices. A child may have the embodied experience of motion along a path, but that does not mean they can reliably count on. The gap between embodied metaphor and stable practice is where PEdT enters.

Practical Elaboration by Training: The Pedagogical Bridge

PEdT refers to the contingent, empirical process by which a course of practical experience or instruction enables the acquisition of new abilities (**Brandom2008**). This process is fundamentally unpredictable. You cannot specify in advance exactly what training regimen will work for a particular learner. It depends on context, biology, biography, and a host of factors that can only be discovered empirically.

When I first started teaching, I thought mathematical concepts were “out there” waiting to be transmitted. If I explained clearly enough, students would understand. This was catastrophically wrong. The problem was not clarity of explanation but the contingent nature of learning. What works for one student fails for another. PEdT is the messy, unpredictable, pedagogically intensive process of stabilizing embodied metaphors into reliable mathematical practices.

Consider the practice of counting. It seems trivial to adults, but it requires stabilizing multiple competencies: stable order (the counting words always go in the same sequence), one-to-one correspondence (each object gets exactly one count), and cardinality (the last number said is the size of the set). A child may have the embodied experience of motion along a path, but translating that into the practice of counting requires training.

The genius of Cognitively Guided Instruction (CGI)—the research program developed by Carpenter, Fennema, and colleagues (**Carpenter1999Book**)—is that it recognizes PEdT as fundamental. CGI does not try to transmit formal procedures. Instead, it elicits students’ informal strategies (grounded in embodied metaphors) and provides the feedback loops necessary to stabilize those strategies into reliable practices. The teacher poses problems grounded in familiar contexts, listens to student thinking, and provides just enough scaffolding to help students refine their implicit methods. This is PEdT. It is contingent, context-dependent, and fundamentally pedagogical.

The crucial point is that PEdT cannot be shortcut. The material inferences provided by embodied metaphors do not automatically yield stable practices. Training is required. And because training is contingent, there are no guaranteed paths. This is why teaching is an art, not a science. The “solved pedagogical problem” of teaching basic counting and addition (**Brandom2008**) is solved not because we have an algorithm for it but because generations of teachers have discovered empirically effective training regimens.

Algorithmic Elaboration: The Logic of Strategy Development

Once primitive practices are stabilized through PEdT, a different mechanism becomes primary: Algorithmic Elaboration (AE). This is where Brandom’s framework shines. AE describes the process by which existing practices are reorganized into more efficient, sophisticated strategies (**Brandom2008**).

Crucially, AE is *in principle* predictable. If you have mastered practice P_1 , then you have—in principle—the resources to engage in practice P_2 , provided P_2 can be algorithmically specified in terms of P_1 . The relationship is logical, not contingent. You can work it out “from your armchair” by analyzing the inferential structure of the practices.

Let me give a concrete example. Once a student has stabilized the practice of Counting On (CO)—starting at one number and counting forward—they have the primitive resources to engage in a more sophisticated strategy called Rearranging to Make Bases (RMB). Suppose the problem is $28 + 7$. The CO strategy would count forward from 28: “29, 30, 31, 32, 33, 34, 35.” This works, but it is inefficient.

RMB recognizes that 28 is two away from the base boundary of 30. So it decomposes 7 into $2 + 5$, adds the 2 to reach 30, and then adds the remaining 5: $(28 + 2) + 5 = 30 + 5 = 35$. This strategy is algorithmically elaborated from CO. It uses the same primitive operations (counting forward, stopping at a target) but reorganizes them strategically: calculate the gap to the next base, decompose the addend, re-associate.

The beautiful thing about AE is that it reveals the logic of practice. RMB is not a new skill learned through training. It is a reorganization of existing skills. The relationship between CO and RMB is PP-sufficient (Practice-to-Practice sufficient): mastery of CO provides, in principle, the resources for RMB. Whether a particular student discovers this reorganization depends on many factors, but the logical relationship is there, waiting to be explicated.

This is the power of AE. It allows learners to bootstrap themselves to higher levels of sophistication by making explicit the logical structure implicit in their existing practices. And when this happens, something remarkable occurs: the elaborated practice explicates the original practice. RMB does not just reorganize CO; it reveals what was always happening in CO. When you count forward, you are implicitly associating units across base boundaries. RMB makes this explicit.

Brandom calls this an LX (Elaborated-Explicating) relationship (**Brandom2008**). The elaborated practice P_2 is not just more efficient than P_1 ; it allows you to *say* what you were implicitly *doing* when you deployed P_1 . This is what mathematics

4.5. *DISTINGUISHING TWO MODES OF PRACTICE DEVELOPMENT: PEDT AND AE*⁹⁷

teachers call “conceptual understanding.” It is not a separate thing from procedural skill. It is the ability to articulate the logic of your own practices.

The Developmental Trajectory: From Embodiment to Explication

Now I can articulate the full developmental trajectory:

1. **Embodied Metaphor:** Mathematical concepts originate in sensory-motor experience. The 4Gs provide material inferences grounded in bodily practice.
2. **Practical Elaboration by Training (PEdT):** Through contingent, pedagogical processes, these material inferences are stabilized into reliable primitive practices. This is what CGI methods facilitate. The embodied understanding of motion along a path becomes the stable practice of Counting On.
3. **Primitive Practices:** Once stabilized, these practices become the foundation for further development. They are not yet “mathematical” in the formal sense, but they are reliable doings.
4. **Algorithmic Elaboration (AE):** Existing practices are reorganized into more sophisticated strategies. This is a logical, in-principle-specifiable process. CO becomes RMB; basic decomposition becomes distributive reasoning.
5. **LX Relationship (Conceptual Understanding):** The elaborated practice explicates the original practice. The student can now say what they were doing, articulating the logic that was always implicit.

Mathematical education often fails when it tries to skip steps, attempting to teach the algorithm (AE) before students have stabilized the primitive practices (PEdT), or assuming that because a student has the embodied metaphor, they automatically have the practice. Both assumptions are wrong.

Mathematical development is not linear. Students move back and forth between metaphors, between practices, between levels of explication. But the underlying structure is there: embodiment → training → primitives → elaboration → explication. Understanding this structure does not give me a guaranteed teaching method (because PEDT is contingent), but it does help me recognize where a student is and what they need next.

Why This Matters for Critical Mathematics

The PEdT/AE distinction has profound implications for how we think about mathematical knowledge and mathematics education. It reveals that mathematics is not a body of pre-existing truths waiting to be discovered. It is a developmental process that begins in the body, requires pedagogical stabilization, and culminates in logical explication.

This is emancipatory. It means students are not empty vessels to be filled with formal procedures. They already have mathematical understanding in their embodied metaphors. The teacher's job is not to transmit but to help stabilize, to provide the contingent training that allows embodied inferences to become reliable practices. And once those practices are stable, the logic of elaboration is already there, waiting to be discovered.

It also means that errors and misconceptions are not defects to be eliminated but potential sites of growth. When a student's strategy fails, it reveals a limit of their current practice. This is an opportunity for PEdT (stabilize a new practice) or AE (reorganize existing practices). The incoherence is not a problem; it is the engine of development.

Finally, it reveals that "conceptual understanding" is not mysterious. It is the ability to articulate the logic of your own practices. When you can say what you are doing—when your practice stands in an LX relation to itself—you have moved from implicit mastery to explicit understanding. This is what it means to *know* mathematics, not just to *do* it.

4.6 Second-Person Reflection: Pragmatic Expressive Bootstrapping

The concept of *pragmatic expressive bootstrapping* provides another avenue for understanding the dialectical movement of mathematical and conceptual history (Brandom2008). It describes the process by which new, more expressively powerful vocabularies emerge from implicit, embodied practices already mastered by practitioners. This is the mechanism by which a conceptual system, in a sense, pulls itself up by its own bootstraps, making explicit what was previously only latent within its own doing.

This process mirrors the structure of *diagonalization* explored in Chapter 7. In diagonalization, a system (like a list of numbers or a set of axioms) is used to construct a new element that is demonstrably part of the system's potential but lies

4.6. SECOND-PERSON REFLECTION: PRAGMATIC EXPRESSIVE BOOTSTRAPPING99

outside its initial, explicit enumeration. Similarly, in pragmatic expressive bootstrapping, a set of existing practices—embodied “knowing-how”—contains the implicit resources to make explicit a new vocabulary that transcends the expressive power of the language used to describe those practices. The key components:

- *Pragmatism*: The foundation is in discursive *practices* and *abilities*. It begins not with abstract rules or meanings, but with the concrete, embodied doings that constitute engagement with the world and each other. This aligns with the theme that the lived, felt experience of mathematics is prior to its formalization.
- *Expressivism*: The function of the new, bootstrapped vocabulary is *expressive*. It does not invent new content out of thin air, but rather makes explicit the inferential norms and commitments that were already implicit in shared practices. It is the process of language recollecting the structure of its own use.

Brandom formalizes this relationship through his Meaning-Use Analysis, which distinguishes between a vocabulary being sufficient to *specify* a practice (*VP-sufficiency*) and a practice being sufficient to *deploy* a vocabulary (*PV-sufficiency*). Pragmatic expressive bootstrapping occurs when a vocabulary (V') is VP-sufficient to specify practices (P) that are, in turn, PV-sufficient to deploy another, more expressively powerful vocabulary (V) (**Brandom2008**). The “bootstrapping” happens when the specifying vocabulary (V') is weaker than the deployed vocabulary (V).

A prime example is the emergence of logical vocabulary. The practices of making and assessing ordinary material inferences (e.g., “Pokey is a dog, so Pokey is a mammal”) are implicit in any autonomous discursive practice. These practices themselves do not require logical vocabulary. However, these doings are sufficient to allow for the introduction of logical vocabulary, such as the conditional (“if...then...”), which then makes those material inferential commitments explicit. The logical vocabulary is thus *elaborated from* and is *explicative of* the underlying practices. Brandom calls this an “LX” vocabulary (**Brandom2008**).

This is significant for a critical mathematics. Pragmatic expressive bootstrapping reveals that the abstract structures of mathematics and logic are not alien, Platonic forms imposed upon the practitioner. Instead, they are the result of discursive practices recollecting themselves, making their own implicit structures explicit. This is an emancipatory understanding. It reframes mathematical knowledge not as a static body of truths to be passively received, but as a dynamic,

ongoing process of conceptual creation. It articulates how learners, through their implicit practices, generate the tools that allow them to transcend their current understandings, turning the implicit ground of embodied rationality into the explicit, shared world of mathematical reason.

4.7 Integration: Modeling Conceptual Change

My work has not only theorized this process but implemented a model of it. The `more_machine_learner.pl` script provides a computational model of a system engaging in normative critique and bootstrapping to overcome its own expressive limitations. Let's examine its operation, which unfolds in four distinct phases.

Phase 1: The Initial Normative Stance

The system begins with a set of norms equivalent to arithmetic over what Lakoff & Núñez call “object collections” (Lakoff2000), which corresponds to the domain of Natural Numbers ($\mathbb{N}$). Within this framework, subtraction is limited. One can only subtract a smaller collection from a larger one. This limitation is formalized as an incoherence axiom in the system's logic (domain $\mathbb{N}$ ‘n’):

An assertion that subtraction resulting in a non-natural number exists is incoherent in the domain of Natural Numbers.

This rule explicitly defines the boundaries of the current conceptual scheme: the idea of $3 - 5$ is, for this system, nonsensical.

Phase 2: The Normative Critique

The dialectical moment is initiated when the system is presented with an external observation that violates its norms. For example, it observes the fact of `minus(3, 5, -2)`. The system's first step is to assess this observation against its current commitments, correctly judging it to be incoherent. Rather than simply rejecting it as “false,” the system recognizes a fundamental clash. It then enters a critique phase to diagnose the precise nature of this incoherence, identifying that the problem is not with the specific numbers, but with the very structure of subtraction in its current domain.

Phase 3: The Normative Shift (Bootstrapping)

The identification of the specific limitation allows the system to propose a solution. It suggests a *normative shift* to a more expressively powerful domain: the Integers ($\mathbb{Z}$, represented as domain ‘z’), where the limitation on subtraction is removed. This is the “bootstrapping” move. The system uses the failure of its old vocabulary (V' : arithmetic over $\mathbb{N}$) to specify a practice (P : subtraction) that is sufficient to deploy a new, more powerful vocabulary (V : arithmetic over $\mathbb{Z}$). The new domain is not arbitrary; it is precisely the domain required to make sense of the practice that broke the old one.

Phase 4: Coherence in the Elaborated System

After adopting the new normative framework by changing its domain to ‘z’, the system re-assesses the original observation. The operation `minus(3, 5, -2)` is now provably coherent. The paradox has been resolved not by dismissing the observation, but by transforming the conceptual framework. The system has pulled itself up by its own bootstraps, using an incoherence generated within its own practices to explicate and adopt a more sophisticated set of norms.

This computational example demonstrates that the dialectical movement from one conceptual stage to the next can be understood as a rational, rule-governed process of critique and repair, turning expressive limitations into engines for conceptual growth. The formalism may help convince those who insist mathematical reasoning *should* be monotonic that it isn’t. The rule that $a - b$ only works when $b \leq a$ is good until it isn’t. Materiality in mathematics proper need not show up as frequently as in elementary classrooms. Instead, it may show up in the ‘grown-up’ world navigated by mathematicians as the bridge between formal systems.

4.8 Conclusion: History as Becoming

This chapter has sought to demonstrate that the history of mathematics, and by extension conceptual history more broadly, can be understood as a rational process of algorithmic elaboration. By moving from the implicit practices of counting, measuring, and inferring, the explicit, formal systems of arithmetic and logic are elaborated. The reconstruction of Euclid’s proof using incompatibility semantics shows how even ancient mathematical insights can be understood as the formal articulation of underlying inferential commitments. The process reveals that a

claim to completeness is self-defeating, not merely false, thereby opening the door to the *infinite*.

However, the journey of knowledge is not a simple, linear unfolding. The limitations of algorithmic elaboration point to the necessity of dialectical ruptures that require a more profound form of conceptual change. Brandom's pragmatic expressive bootstrapping provides a model for these moments. It shows how a system of practices can, in a move analogous to diagonalization, generate a new, more expressively powerful vocabulary that makes its own implicit normative structure explicit. This is not magic, but language unfolding itself. It reframes mathematical knowledge not as a static collection of truths, but as a dynamic, self-correcting, and emancipatory process: a constant becoming.

By understanding the rules implicitly followed, it becomes possible to make them explicit, critique them, transcend them, or identify with them. The computational model demonstrates that this dialectical movement can be formalized as a rational process of normative critique and repair. The normative shift from $\mathbb{N}$ to $\mathbb{Z}$ exemplifies how mathematical systems contain within themselves the resources for their own transformation when confronted with observations that are incoherent under their current norms.

This understanding of mathematical history as becoming rather than being carries profound implications for mathematics education. Students need not ~~vision~~ mathematics as a finished edifice to be passively received but as a living practice in which they participate. The errors and misconceptions that arise in learning are not mere defects to be eliminated but potential sites of conceptual growth, incoherences that can drive the elaboration of more powerful understandings.

The initial misrecognition—that literature reviews capture a complete totality, that algorithms are mere mechanical procedures, that mathematical history is a static accumulation of facts—is now partially understood as a failure to grasp the dynamic, self-elaborating nature of conceptual development. “The Literature” is not a fixed object but an ongoing conversation. Algorithms are not arbitrary procedures but rational chains of material inference. Mathematical history is not a dead archive but a process of becoming through which practices make their own implicit structures explicit.

This partial understanding opens new questions: How do these processes of algorithmic elaboration and pragmatic expressive bootstrapping apply to the specific domain of arithmetic operations? If numerals are first-person pronouns recollecting the null representation, how do addition, multiplication, and other operations emerge as normatively regulated practices? The next chapter, “Operation,” will explore how mathematical operations themselves can be understood as embodied,

normatively regulated practices that emerge from the triadic unity of subjective experience, intersubjective norms, and objective structures.

Chapter 5

10: Operation

Abstract

This chapter explores the nature of mathematical operations, arguing that they are not merely formal manipulations of symbols but rather expressions of embodied, normatively regulated practices. Building on the previous chapters' reconstruction of numerals as pronouns and the history of mathematical development through algorithmic elaboration, the analysis develops a framework for understanding what the author calls “critical arithmetic”—a pre-formal system that captures the dynamic, error-inclusive nature of everyday arithmetic. Through the lens of multiplication, the chapter demonstrates how operations emerge from the interplay of three dimensions: embodied practices, inferential rules, and formal structures. The discussion begins with the embodied basis of arithmetic, using the example of a strategy called C2C (Coordinating Two Counts by Ones) documented in Cognitively Guided Instruction research. The chapter then articulates how subjective experiences are transformed into shared, rule-governed practices through a normative framework based on Robert Brandom's incompatibility semantics and the embodied cognition research of George Lakoff and Rafael Núñez. Finally, the analysis demonstrates how these practices give rise to stable, objective procedures that can be modeled as formal automata. Throughout, the chapter emphasizes that mathematical operations are grounded in human activity and social normativity rather than existing as timeless, abstract truths.

5.1 The Misrecognition of Operation

The preceding chapters reconstructed the nature of number itself, beginning with a student's profound question asked in a moment of grief and frustration: "Mr. Savich, what even is two?" That moment led away from the conventional view of numbers as abstract, pre-existing objects and toward an account of them as first-person recollective pronouns. Numerals, symbolized by the null representation $\emptyset$ (Chapter 8), emerge from the dynamic interplay of linguistic self-reference, anaphorically recollecting the enabling conditions of thought—the transcendental "I think".

Having explored what numbers *are*—reflections of dynamic, self-recollecting human being—this chapter now turns to the equally fundamental question of what it means to *do* things with them. Conventional accounts treat mathematical operations as formal manipulations of abstract symbols, governed by axioms that are either arbitrary conventions or self-evident truths. In this view, addition, subtraction, multiplication, and division are processes applied to numbers, with rules to be memorized and followed.

I propose a different understanding, continuous with the emancipatory project of this book: mathematical operations are not arbitrary conventions but are themselves expressions of embodied, normatively regulated practices. The "laws" governing the use of numbers are best understood not as transcendent truths, but as algorithms or inference chains—normatively regulated doings that emerge from embodied engagement with the world and with others. This misrecognition—treating operations as mechanical procedures divorced from human meaning-making—parallels the earlier misrecognition of numbers as abstract objects divorced from the thinking subject who deploys them.

This chapter articulates inference rules for what I call "critical arithmetic"—an open-ended, pre-formal system that captures the kind of arithmetic used in everyday life—errors and all. Drawing on the empirical foundation provided by Cognitively Guided Instruction (CGI) research (**Carpenter1999**), which established that children possess rich, informal mathematical knowledge long before formal instruction, the investigation reveals how mathematical operations unfold in lived experience.

5.2 The First Determinate Negation: Critical Arithmetic and the Triad

To make this misrecognition determinate, I introduce the concept of *critical arithmetic*. This framework ~~negates~~ the conventional view by denying that operations exist as pre-given formal procedures. Instead, critical arithmetic reveals operations as practices that emerge through the triadic structure of validity developed throughout the inquiry.

The investigation into student-invented strategies is deeply indebted to the paradigm shift initiated by Cognitively Guided Instruction (CGI). Emerging in the 1980s from research by Carpenter, Fennema, and colleagues, CGI established that children possess rich, informal mathematical knowledge long before formal instruction (**Carpenter 1999**). This research provided the empirical foundation for the constructivist turn in mathematics education, asserting that students actively construct understanding by solving problems in meaningful ways, rather than passively receiving algorithms.

However, while constructivism and CGI have demonstrated significant positive effects on students' conceptual understanding, it is burdened by privileging subjective-validity claims over normative and objective validity claims. I want to use the brilliant results that emerged from the careful methods of CGI and constructivism, but reframe some of that work in a way that better integrates the triadic framework of validity developed in this book.

The primary 'revision' is that strategies are not precisely 'invented.' They emerge as intersubjectively constituted practices within a social context. As dyadic communicative *actions* – a speaker stakes their identity to their expressions under the assumption that someone *might* be able to understand them – they are structured by action-impeti, self-monitoring feedback loops, and existential needs for recognition. They are like orbits in intersubjective space around self-certainty, the 'gravitational well'/singularity/hole in representational space.

This revision incorporates Derridean insights into the concept of invention. At its core, invention is paradoxical: the invention must be recognized as different from what has come before, it must also be intelligible within the existing framework of understanding.

My revision also challenges the preferred order of explanation that constructivists often employ. The tendency is to value constructed knowledge as more valid than memorized procedural knowledge. I don't disagree with that valuation, but it isn't how I tend to learn. Instead, I tend to read/hear/observe, experience

the I-feeling, as if I am genuinely recognized by what I hear, reflect, realize I have no idea what they mean, try to fit them together like a puzzle with how I understand the world, experience the I-feeling again when I take the new idea on as a commitment, and then realize the new idea is not self-certainty.

That abstract description could stand some nuance. Take counting. A child usually learns the chant of 1, 2, 3, etc. They aren't usually counting anything, they're just repeating the chant. This is memorization, not construction. But it is a necessary step. The child must have the words in their head before they can use them to count.

Then, through practice and social interaction, the child begins to understand that these words correspond to quantities of objects. They start to count real things, experiencing the I-feeling of successfully matching the number of objects to the last number said. This is the construction phase, where the child actively builds understanding.

Then, eventually, they might experience the paradox of identity encoded in the claim that ten ones is one ten. This is first experienced as a falsity: ten pennies is *not* one dime. There's more stuff, so they can't be the same! Then they might actually synthesize the concept of zero, realizing that, in intersubjective practices, people really do trade in ten pennies for one dime. Or they might learn to decompose a base with counting cubes. All the while, I imagine children are feeling the same sort of existential fear tracks me, like a prowling shadow, whenever I try to learn something new. Am I okay??

In attempting to connect the various strategies of CGI to embodied practices, bootstrapping from counting to division, the purpose is not to suggest that people actually learn so systematically. The curricula doesn't start with the breath. It started, for me, when my sister, who is two years older than I am, came home from school talking about the books she was reading. I *wanted* to be as cool as she is, so I pretended to know how to read. I pretended to know how to use fractions. I pretended a lot. Eventually, I became firmly committed to communicative norms associated with mathematics. I learned to read, I learned to use fractions, and use variables. But I still don't understand why, for example, $4 \frac{m}{s^2} \times 2s = 8 \frac{m}{s}$ – why an acceleration multiplied by a unit of time gives a velocity. I mean, I can explain such equations using an incredibly rich vocabulary in a way that might convince a skeptic and convince anyone who doubts my understanding that I actually do understand it. But the body-feeling of acceleration accompanied by a few seconds of sitting around waiting, doesn't feel like a velocity. The reification of body-feelings as anaphoric expressions doesn't easily backfill into those same body-feelings. I don't experience speed, for example, as a body-feeling at all. Do you?

You must understand that the current wisdom suggests that the solar system is orbiting the center of the Milky Way at 514,000 miles per hour. Do you *feel* that? No. We feel differences—jolts in the catastrophe machine. So how would it be the case that two experientiable concepts, acceleration and waiting, multiply to give an unexperientiable concept of velocity? The best I can say is that I'm not sure.

In any case, formalizing student-invented strategies provides some benefits. By making them explicit in a vocabulary built to describe speech-actions (formal automata), commonalities can be discerned. States and transitions may be shared across different operations in ways that the rigid adherence to the communicative norms of constructivism do not afford, precisely because of their commitment to human-centered learning. I want to be able to leverage the massive new expressive powers of computational systems to explore the space of possible strategies. This is not to say that I want to replace human-centered learning with machine-centered learning. Quite the opposite. I want to use machines to explore the space of possible strategies so that I can better understand how humans learn.

A computer, for example, can systematically explore the space of possible numbers to use in examples that might be used to invoke the crisis of strategic thinking. Right now, there are probably thousands of children working on computer-generated worksheets where the numbers being used are almost arbitrary. Yet math teachers will recognize that some numbers are more 'natural' than others for teaching a concept. I liken this benefit to the problem of protein-folding. Consider how many medicines utilize proteins that were discovered by accident. Since the mid-2000s, the problem of protein-folding has been well-studied, but now machines have the capability to assist in this research, systematically exploring the space of possible proteins and their interactions. With AI, it seems possible to systematically explore the space of possible numbers to construct exemplary examples that can be used to bootstrap students' understandings into new operations. The next 'phase' of math education research may include some such computationally intensive explorations.

Consider the following example. Imagine a first-grader who can count-on, but who can't yet rearrange to make bases. Suppose they learned to count to 100 in kindergarten, and the curriculum is designed to use that prior knowledge. A computer-generated worksheet might draw from the nearly 200 million partitions of 100 to randomly select some problems. It presents the child with $15 + 2$. The child learns nothing, because they can count-on. Another child is randomly presented with the problem $15 + 7$. They are too 'lazy' (in a good way) to want to do 7 inferential steps. So, instead, they draw on the material inference for object-collections that implicitly involves associativity and commutativity, to move 3

ones from 15 to 7, resulting in $12 + 10$, and the notion that 1 base + 1 base is 2 bases, to obtain 22. While ‘inventing’ such a strategy takes real effort, the new structure for addition doesn’t require as many monotonous inferential steps. Testing the strategy to ensure it always works requires more steps, but once the strategy has been tested, what might have taken 7 steps now only takes 3. Ideally, a teacher would serve rich problems to students, being able to tell what sorts of examples would be most likely to invoke the crisis of strategic thinking. But practically, teachers are busy. We’re already letting machines do a lot of the heavy lifting in math education, but the machines are relatively stupid. They have not been trained to think about mathematics like people think about mathematics.

These challenges motivate the need for a more structured understanding of the landscape of student-invented strategies. The formalization project undertaken here, exemplified by the Hermeneutic Calculator (**savich_hermeneutic_nodate**), is not intended to restrict student invention but to rigorously analyze the strategies that emerge within intersubjective space. By treating these strategies as computational choreographies, we can move beyond the limitations of isolated teaching experiments and discover relationships that might otherwise remain obscured.

The triadic framework examines operations from three integrated subject positions: (1) The **subjective validity** of embodied practices, grounded in first-person lived experience; (2) The **normative validity** of inferential rules, structured through second-person commitments that exclude error; (3) The **objective validity** of formal structures, modeled as third-person automata. These three dimensions interpenetrate, forming a unity where the formal emerges from the felt, the abstract from the concrete. Operations stabilize around points of self-certainty where subjective experience, normative regulation, and objective structure achieve coherence.

5.3 Systematic Analysis: The Recipe from Embodiment to Norms

The triadic progression from subjective to objective requires a systematic recipe for moving from private, embodied feeling to shared, communicable, and rule-governed practice. This section articulates that recipe using multiplication as a running example, demonstrating how the formal emerges from the felt.

The Embodied Ground: Subjective Validity

The journey of any operation begins not with a symbol, but with a doing. Children, before they are ever taught a formal multiplication algorithm, invent their own strategies grounded in embodied experience. Consider one of the most intuitive strategies: *Coordinating Two Counts by Ones* (C2C). To find the total number of items in 3 groups of 4, a child counts: “one, two, three, four” for the first group, then continues “five, six, seven, eight” for the second, and “nine, ten, eleven, twelve” for the third. All the while, a second, implicit count tracks the number of groups processed.

This is a lived, felt process—a rhythmic activity grounded in the basic human capacities for grouping objects, maintaining a sequence, and tracking multiple streams of information. This represents the *subjective* pole of the operation. It is a form of knowing that is inseparable from the physical act of doing, a claim to subjective validity: “this is how it feels right to me to do this.”

From Embodiment to Norms: The Recipe

How do we move from this private feeling to a mathematical operation? The recipe unfolds in systematic steps:

Step 1: Start with the Embodied Practice. Begin with an intuitive strategy like C2C, grounded in subjective validity.

Step 2: Articulate Material Inferences. Re-describe this practice as a set of material inferential commitments using incompatibility semantics (**Brandon2008**). The “goodness” of an operation is not an abstract truth but a normative status conferred upon a practice within a recognitive community. The operation $3 \times 4 = 12$ becomes a material inferential commitment:

$$\{3 \text{ groups of } 4\} \models_I \{12\}$$

Using the definition of incompatibility entailment, this means that everything incompatible with the result ‘12’ (e.g., asserting the result is 11 or 13) must also be incompatible with the initial premise ‘3 groups of 4’. By systematically excluding what is incoherent, we define the boundaries of coherent practice.

Step 3: Transpose Embodied Metaphors. Drawing on Lakoff and Núñez (**Lakoff2000**), transpose their four grounding metaphors into the modal logic of Chapter 1. *Arithmetic as Object Collection* grounds commutativity and associativity in the physical indifference of pooling. *Arithmetic as Motion Along a Path* grounds subtraction in backward movement. These are not mere mappings but

phenomenological performances—rhythms of temporal compression (reification [T]) and decompression (sublation [LG]).

Step 4: Generate the Formal Automaton. The result is a stable, repeatable procedure modeled as a Finite State Automaton. The automaton embodies the norms of the practice, providing a third-person, formal description. Its validity is alethic: when followed, it correctly yields results consistent with the normative framework. This is the form of the practice made explicit, a determinate structure.

5.4 The Dialectical Turn: Choreography and Fractal Geometry

Mathematical operations are, fundamentally, choreographed movements of thought. Each arithmetic strategy represents what I term “written choreography for embodied cognition” (**savich_hermeneutic_nodate**). A formal automaton becomes a script for the temporal unfolding of mathematical understanding: initialize, transform, check, recurse, terminate. This framing preserves *how* a student actually moves through a calculation—counting up, pausing at a boundary, decomposing a number—rather than replacing those movements with opaque symbolic shortcuts.

The dialectical turn emerges from recognizing that this choreography exhibits genuine computational self-similarity. Across twenty-five formalized student strategies in the Hermeneutic Calculator project, I consistently recover the same fractal pattern. Each strategy exhibits an **iterative core**—a minimal loop of initialize, step, and condition check—nested within a **strategic shell** that prepares, optimizes, or transforms the problem so the core runs with fewer or cognitively lighter iterations.

Consider the *Sliding* strategy for subtraction. Faced with $74 - 36$, a student might recognize that both numbers can be adjusted: $(74+4) - (36+4) = 78 - 40 = 38$. This emerges from the *Motion Along a Path* metaphor, where distances remain invariant under translation. The formal automaton reveals its computational structure: the strategic shell identifies a convenient adjustment value K that simplifies calculation (creating a multiple of ten), then invokes the iterative core to perform the adjusted subtraction.

Strategies such as Rearranging to Make Bases, Sliding, and Distributive Reasoning wrap the iterative core with analysis that computes gaps, splits factors, or slides both numbers to maintain invariant relationships. Because the shell often invokes the core as a subroutine, the global structure becomes genuinely self-similar:

strategies contain and sometimes nest earlier strategies, producing a computational fractal rather than mere metaphorical flourish.

Two fundamental movements underlie all arithmetic strategies. First, **temporal compression** (sublation/recollection) unitizes many micro-acts into larger cognitive units: ten ones become one ten, three base jumps become a single composite stride. Second, **temporal decompression** (determinate negation) strategically expands a composite to restore fine control: borrowing a ten, splitting five into two plus three, decomposing a factor for distributive reasoning. Mathematical fluency emerges as students learn to coordinate these movements, developing the capacity to know *when* to expand and *when* to re-compress.

5.5 Second-Person Reflection: Elaboration and Formalization

The distinction between Practical Elaboration by Training (PEdT) and Algorithmic Elaboration (AE) is crucial for understanding the structure of mathematical development. This distinction explains how mathematical understanding moves from contingent pedagogical stabilization to necessary logical elaboration.

Practical Elaboration by Training (PEdT) is the contingent, pedagogical process emphasized in Cognitively Guided Instruction that transforms implicit material inferences into explicit, teachable procedures. The embodied origins of mathematical concepts—the four grounding metaphors of Object Collection, Object Construction, Measuring Stick, and Motion Along a Path—require pedagogical stabilization to become reliable mathematical practices. Students must learn to count on consistently, to decompose numbers strategically, and to recognize when particular approaches are appropriate. This stabilization is fundamentally pedagogical—it depends on teaching, practice, and the social regulation of mathematical activity.

Algorithmic Elaboration (AE), by contrast, represents the logical restructuring of existing abilities rather than their mere strengthening through repetition. When students reorganize their Counting On practice into the more sophisticated Rearranging to Make Bases strategy, they are not simply becoming faster counters but are discovering new inferential relationships. The elaborated strategy makes explicit what was implicit in the original practice: that numerical relationships can be preserved under certain transformations.

Strategies that stand in an LX (Elaborated-Explicating) relationship—where

later practices both derive from and make explicit earlier ones—constitute what mathematics educators recognize as “conceptual understanding.” The Rearranging to Make Bases strategy explicates the associative structure latent in Object Collection. Distributive Reasoning articulates the multiplicative structure implicit in repeated addition. These relationships demonstrate how formal mathematical properties emerge from and remain grounded in embodied practice.

This second-person reflection reveals that mathematical development is not a smooth, continuous progression but involves dialectical moments where existing practices must be transcended. The failure of a strategy under certain conditions (e.g., the limitation of subtraction in $\mathbb{N}$ when facing $3 - 5$) creates the incoherence that drives elaboration toward more expressive frameworks (the integers $\mathbb{Z}$). Only after PEdT stabilizes primitive practices can AE restructure them into sophisticated strategies. The structure of mathematical development thus exhibits both contingent pedagogical moments and necessary logical transformations.

Formalizing Algorithmic Elaboration: Counting On to Rearranging to Make Bases

The claim that Rearranging to Make Bases (RMB) algorithmically elaborates Counting On (CO) can be demonstrated formally through automata specifications. By examining the computational structure of each strategy, we can see precisely how the elaborated practice reorganizes and nests the primitive practice.

The Primitive Practice: Counting On

Counting On is the foundational additive strategy. To solve $A + B$, the student starts at A and counts forward B times: “ $A + 1, A + 2, \dots, A + B$.” This embodies the Motion Along a Path metaphor, where numbers are locations and addition is forward movement.

Formally, Counting On can be modeled as a deterministic pushdown automaton (DPDA) with a bounded stack representing base-10 place values. The machine maintains units, tens, and hundreds on the stack, incrementing and carrying as needed.

Table 5.1: *Counting On Automaton (M_{count})*. The formal specification treats counting as iterative unit increments with automatic carry propagation (sublation) across place values. Each “tick” input advances the count; overflow at U_9 triggers temporal compression into the tens place.

Current State	Input	Top of Stack	Next State	Action (Stack)	Interpretation
q_{start}	ε	Z_0	q_{idle}	Push(U_0, T_0, H_0)	Initialize to 0
q_{idle}	tick	U_n ($n < 9$)	q_{idle}	Pop; Push(U_{n+1})	Increment units
q_{idle}	tick	U_9	q_{inc_tens}	Pop	Unit overflow $\rightarrow$ carry
q_{inc_tens}	ε	T_m ($m < 9$)	q_{idle}	Pop; Push(T_{m+1}, U_0)	Increment tens, reset units
q_{inc_tens}	ε	T_9	q_{inc_hund}	Pop	Tens overflow $\rightarrow$ carry
q_{inc_hund}	ε	H_k ($k < 9$)	q_{idle}	Pop; Push(H_{k+1}, T_0, U_0)	Increment hundreds
q_{inc_hund}	ε	H_9	q_{halt}	Pop; Push(H_0, T_0, U_0)	Counter overflow

The choreography of Counting On is simple but profound. Each “tick” is a bodily rhythm—a finger tap, a verbal count, a step forward. The carry operation is temporal compression: ten unit steps recollected as one ten. This primitive practice is reliable, but inefficient for larger addends.

The Elaborated Practice: Rearranging to Make Bases

Consider my favorite example, rearranging to make bases. Remember: the formalization is not meant to intimidate—it is built to break. As my understanding of formal systems grows, the vocabularies I use to express the pragmatics of mathematical strategic thinking, the formalizations change. There is no claim to finality or completeness.

The video I analyzed is from **Carpenter1999**<empty citation>. The strategy descriptions and examples were adapted from Amy Hackenberg’s careful reconstruction of those videos to teach pre-service teachers **HackenbergCourseNotes**<empty citation>.

- **Teacher:** Lucy is eight fish. She buys five more fish. How many fish will Lucy have then?
- **Sarah:** 13.
- **Teacher:** How’d you get 13?

- **Sarah:** Well, because eight plus two is ten, but then two plus three is five. And she wants to buy five more fish. So you take care of two, and you need to add three more. And so I add three more, and you get 13.

Notation Representing Sarah's Solution:

$$8 + 5 = \square$$

$$8 + 2 = 10$$

$$2 + 3 = 5$$

$$8 + 5 = 10 + 3$$

$$8 + 5 = 13$$

Description of Strategy:

Objective: Rearranging to Make Bases (RMB) means shifting the extra ones from one addend over to the other so that one of the numbers becomes a complete multiple of the base (a whole “group” of that base). This rearrangement simplifies the addition process because there are established patterns for adding an exact multiple of the base. In other words, when you add a full group of base units to a number, the ones digit stays the same while only the digit representing the base (like the tens place) increases.

Rearranging to Make Bases transforms the counting procedure by introducing strategic planning. Instead of counting forward B times, RMB recognizes that reaching the next base boundary (10, 20, 100, etc.) creates a cognitive landmark. The strategy decomposes B into two parts: the gap K needed to reach the next base, and the remainder R . Symbolically: $A + B = (A + K) + R$, where $A + K$ is a clean base.

For example, solving $28 + 7$: the gap from 28 to 30 is 2, so decompose $7 = 2 + 5$, yielding $(28 + 2) + 5 = 30 + 5 = 35$. The student has reorganized the primitive counting procedure to minimize cognitive load.

Table 5.2: *Rearranging to Make Bases Automaton* (M_{RMB}). This strategy algorithmically elaborates Counting On by adding a strategic shell that computes the gap K to the next base, decomposes the addend B , and recombines. The core counting operations are nested within this higher-level strategic structure.

Current State	Condition	Next State	Action	Interpretation
q_{start}	—	q_{calcK}	$K \leftarrow 0; A_{temp} \leftarrow A$	Initialize
q_{calcK}	$A_{temp} < \text{NextBase}(A)$	q_{calcK}	$A_{temp} \leftarrow A_{temp} + 1;$ $K \leftarrow K + 1$	Count up to find gap K
q_{calcK}	$A_{temp} == \text{NextBase}(A)$	q_{decomp}	$A' \leftarrow A_{temp}$	Gap found, store new base A'
q_{decomp}	$K > 0$	q_{decomp}	$B \leftarrow B - 1;$ $K \leftarrow K - 1$	Decompose B by transferring K
q_{decomp}	$K == 0$	q_{recomb}	$R \leftarrow B$	Remainder R is what's left of B
q_{recomb}	—	q_{accept}	Output $A' + R$	Combine base + remainder

The Logic of Elaboration

Comparing these two specifications reveals the precise mechanism of algorithmic elaboration. RMB does not replace CO; it reorganizes it. The q_{calcK} state uses the primitive counting operation (incrementing A_{temp}) to calculate the gap. The q_{decomp} state uses subtraction (the inverse of counting) to split B . The final q_{recomb} state invokes addition on the transformed problem.

The elaboration is *algorithmic* because it is specifiable in principle: given mastery of counting up and counting down, the RMB procedure can be constructed through sequencing and conditional branching. It is *explicating* because RMB makes explicit what was always implicit in CO—the associative structure of addition. When counting from 28 to 35, the student was implicitly crossing the base boundary at 30. RMB makes this boundary crossing explicit and strategic.

This is the structure of LX relationships throughout arithmetic. Distributive multiplication explicates the iterative structure of repeated addition. Decomposing a base (‘borrowing’) explicates the decomposability of place-value units. Each elaborated strategy reveals the logic that was always present in the primitive practice, transforming implicit knowing-how into explicit knowing-that.

Figure 5.1: *The Fractal and Dialectical Geometry of Arithmetic*. Fractalized Arithmetic as Automata: The fractal scaling of the Iterative Core demonstrates computational self-similarity. The superimposed loops show that the Core’s structure remains invariant while the scale of action changes from units to bases. The dialectical inversion (Red/Purple) visualizes how the same structure operates in opposite orientations.

The fractal architecture illustrated in Figure 5.1 reveals how mathematical strategies achieve their power through self-similar structure. The iterative core—the fundamental loop of initialize, step, and check—scales across different mathematical magnitudes while maintaining its essential form. This scaling property allows strategies to work across place values, demonstrating how mathematical understanding exhibits genuine computational self-similarity. The integration of inversion (Möbius orientation flipping) with fractal nesting (core within shell) explains both the unity and diversity of arithmetic strategies: they are variations on the same underlying mathematical structure.

The formal automata do more than describe strategies; they articulate the developmental logic of mathematical understanding. They show how practices that feel natural to experienced mathematicians emerge through systematic reorganization of primitive embodied doings. They demonstrate that “conceptual understanding” is not a mysterious extra ingredient added to procedural skill but the capacity to articulate the inferential structure of one’s own mathematical practices.

5.6 Integration: The Dialectical Structure of Operations

The concept of inversion is central to the dialectical structure of mathematical operations. The analysis reveals that operations exhibit a fundamentally dialectical relationship where opposing procedures share underlying computational structures. Subtraction strategies emerge not through the invention of entirely new procedures but through the inversion or repurposing of addition approaches.

The Missing Addend strategy reframes subtraction as forward accumulation: “What must I add to 36 to reach 74?” becomes the question structuring $74 - 36$. Counting Back mirrors Counting On, moving in the opposite direction along the number line. The Sliding strategy preserves difference across translation, maintaining invariant relationships under transformation. Borrowing reverses the carry

procedure by decompressing previously sublated place-value units: where carrying compressed ten ones into one ten, borrowing expands one ten back into ten ones.

This dialectical relationship extends beyond the superficial observation that subtraction “undoes” addition. The formal automata reveal that operations often share identical computational cores while differing in their orientations or interpretations. The Möbius strip visualization captures this relationship precisely: the underlying structure (the Green core) remains invariant while the operational orientation flips between additive (Red) and subtractive (Purple) modes.

Figure 5.2: *The Geometry of Inversion: Addition and Subtraction as a Unified Structure.* The Möbius strip visualization articulates the dialectical relationship between addition and subtraction. The automata analysis demonstrates how they represent inverse orientations of the same underlying temporal structure.

5.7 Conclusion: Operations as Creative Possibility

This chapter has articulated a vision of mathematical operations as embodied, normatively regulated practices that emerge from the triadic unity of subjective experience, intersubjective norms, and objective structures. Throughout this investigation, the argument has been that operations are not mere procedures applied to pre-existing objects but are expressions of human being-in-the-world.

The *subjective* experience of doing (e.g., the felt rhythm of Coordinating Two Counts by Ones) provides the initial, meaningful content of an operation. It is a first-person grounding of mathematical thought, rooted in the proprioceptive and temporal structures explored in Chapter 1. The *normative* framework of incompatibility semantics provides the rules of the game—a second-person structure of commitments and entitlements, defining coherence by systematically excluding error. The *objective* structure of the automaton provides the formal, repeatable, and communicable form of the practice—a third-person, explicit representation of a normatively governed activity.

The critical arithmetic I propose is not a closed formal system but an open-ended, pre-formal practice that is continuously evolving through dialogue and critique. It is a practice that values the agency of learners and the creative potential of mathematical thought. It resists the scientific impulse to reduce the rich, messy,

human process of mathematical meaning-making to a single, “correct,” disembodied algorithm.

The Hermeneutic Calculator project demonstrates that student-invented strategies can be formalized without losing their connection to embodied meaning. The automata serve not as replacements for human mathematical thinking but as precise descriptions of the temporal unfolding of mathematical understanding. By treating each strategy as choreographed movement of thought, the formalization preserves rather than eliminates the human dimension of mathematical activity.

Understanding mathematical operations as embodied, normatively regulated practices reveals arithmetic not as a closed system of fixed procedures but as an open field of creative possibility. Students do not simply learn to execute predetermined algorithms but participate in the ongoing elaboration of mathematical understanding. This participation is simultaneously deeply personal—grounded in each individual’s embodied experience—and thoroughly social—regulated through the shared norms of mathematical practice.

The analysis demonstrates how formal mathematical structures emerge from rather than supersede lived experience. The automata that model student strategies are not abstract computational devices but precise articulations of embodied mathematical thinking. They reveal the temporal dynamics through which mathematical understanding unfolds, showing how the formal and the felt, the abstract and the concrete, the subjective and the objective are integrated within a unified account of mathematical meaning-making.

Embracing this vision—seeing operations as a synthesis of subjective feelings, normative commitments, and objective structures—moves mathematics education toward a more emancipatory stance, fostering not just technical proficiency, but also rigorous thinking, creative exploration, and humane flourishing.